

Introduction to Deep Learning

“Neural Networks”

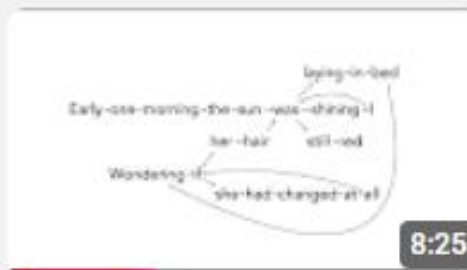
- Probably this is last year NN is given in IMPR – Now covered in other courses
- Only brief coverage (lecture + 2 exercises). More details in Deep Learning courses, New IML (5 weeks!)
- Neural networks are parts of most computer vision systems today (And all other fields...)
- Many previously impossible tasks became possible using Neural Networks

Can We Study Only Neural Networks?

- NN is great on tasks with available examples, and without well defined algorithms: Recognition, Classification, etc.
- A well-defined algorithm or an exact equation are preferred to Neural Network when available
 - Solving Equations, Geometry Computations, Physics Principles
- In most cases NN is part of a system. In addition, Domain knowledge Needed.

Steve Seitz Videos

23



Large Language Models from scratch

Graphics in 5 Minutes • 354K views • 2 years ago

24

the recipe for chocolate quark cake pancakes.

1. Large, generally spherical
 2. Not microscopically
 3. Not highly polished (most
 4. Coloured) (very faint)
 5. Not transparent
 6. Not
 7. The highest degree of
 8. Not microscopically

Directions:

1. Is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
2. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
3. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
4. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
5. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
6. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
7. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
8. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
9. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.
10. It is a complex lipid molecule that is composed of several parts, like a lipid molecule with an oil.

7:19

Large Language Models: Part 2

Graphics in 5 Minutes • 121K views • 2 years ago

25



Squirrel reaching for a nut. Van Gogh paints

6:00

Text to Image in 5 minutes: Parti, Dall-E 2, Imagen

Graphics in 5 Minutes • 13K views • 2 years ago

26

supplement to the
the manuscript.



6:13

Text to Image: Part 2 -- how image diffusion works in 5 minutes

Graphics in 5 Minutes • 10K views • 2 years ago

Machine Learning Example: Visual Classification

A common machine learning task:

Learn from examples a function F so that:

$$F\left(\text{img}\right) = \text{Man}$$

$$F\left(\text{img}\right) = \text{Woman}$$

- Hard to “hand craft” a good approximation to F
- F can be learned by NN from labeled training examples
- F is tested on a different labeled test set

The Machine Learning Framework

$$Y = F(x)$$

The diagram shows the equation $Y = F(x)$ with three red arrows pointing to its components. An arrow points from the text 'Output/ Classification' to the variable Y . Another arrow points from the text 'Prediction Function' to the function symbol F . A third arrow points from the text 'Input/ Image' to the variable x inside the parentheses.

Output/
Classification

Prediction
Function

Input/
Image

- **Training:** Given a training set of labeled examples $\{(x_1, y_1), \dots, (x_N, y_N)\}$, estimate the prediction function F by minimizing the prediction error on the training set.
- **Testing:** Apply F to a never seen before test set x and output the predicted value $y = F(x)$

Training LLMs (Large Language Models)

$$Y = F(x)$$

Next Word Prediction Function Long Text

- **Training:** Given a training set of all the texts available through the internet, estimate the prediction function F of predicting the next word given the prior text. No need for labeling...
- “This sentence is an example for training a” ??????

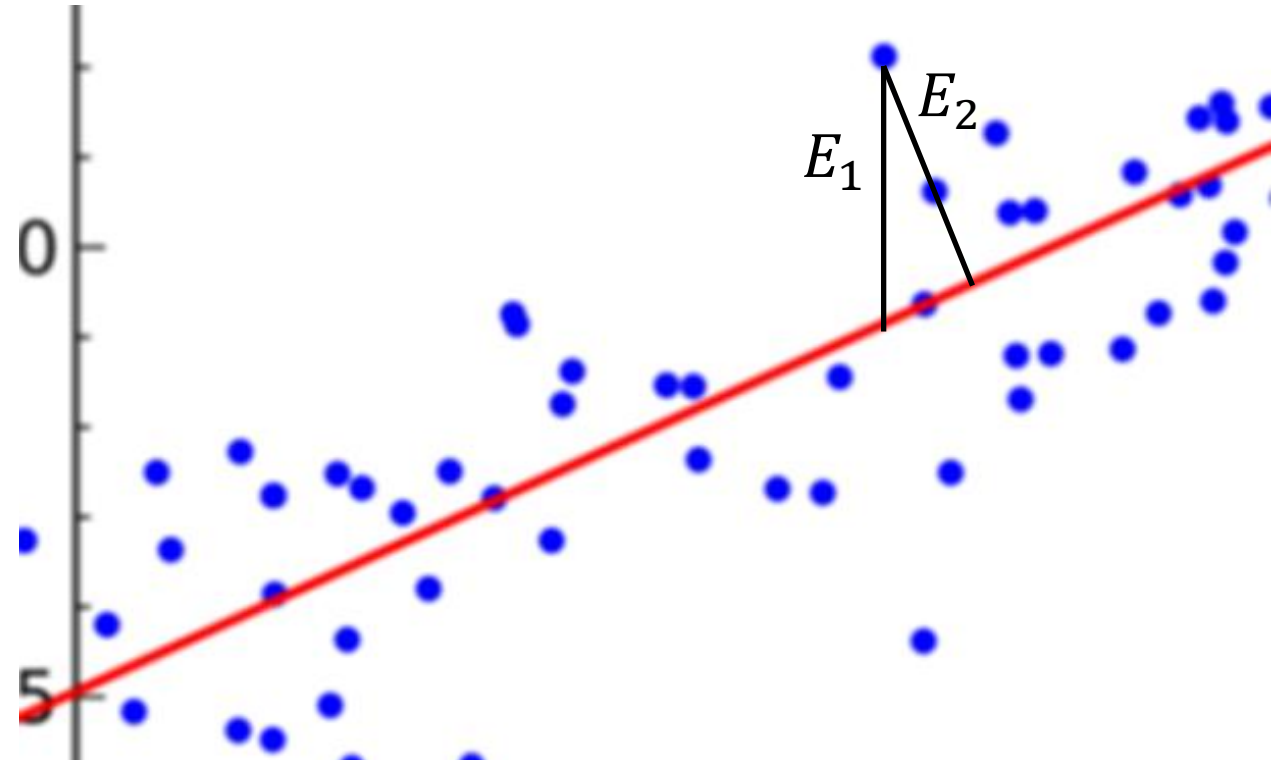
Linear Regression – Learning a Linear Function

- Learn a parameters of a linear function from examples
- E.g. Given a set of examples (x_i, y_i)
- Find (a_0, a_1) giving the best approximation $y(x) = a_0 + a_1 x$
- Minimizing the cost function

$$E_1 = \sum_i (y_i - (a_0 + a_1 x_i))^2$$

- Or from line equation:
 $a_0 + a_1 x + a_2 y = 0$

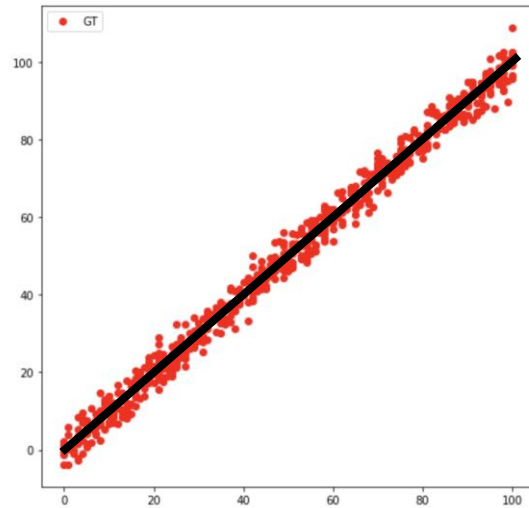
$$E_2 = \sum_i (a_0 + a_1 x_i + a_2 y_i)^2$$



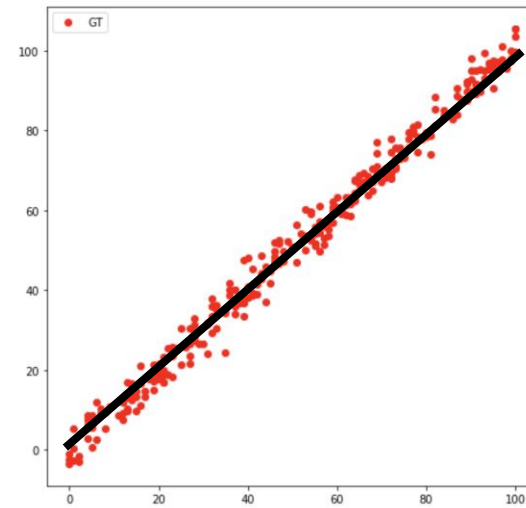
Training and Testing

- Most data (~80%) used for training, and some for testing (~20%).
- Important to see that both training and test errors go down!
- Training error should always go down (minimization on it...)
- Test error will go down if the learned function is generalized beyond the training set

Training



Test



Memorizing vs. Generalization



Training set

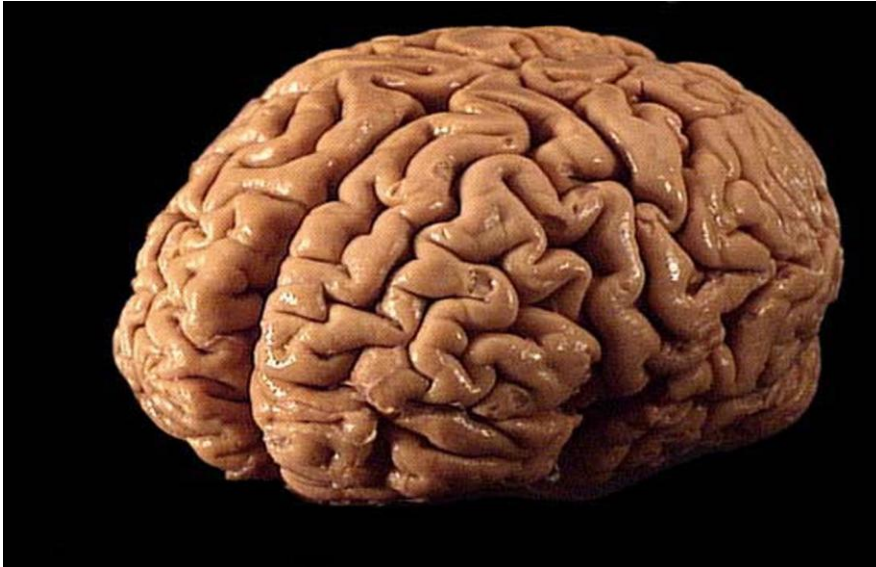


Test set

How well does the learned function extend from the training set to a new test set?

Inspiration for functions to learn: Our Own Neural Network

Human Brain



Average Weight: 1.5 Kg

Average Size: 1260 cm³

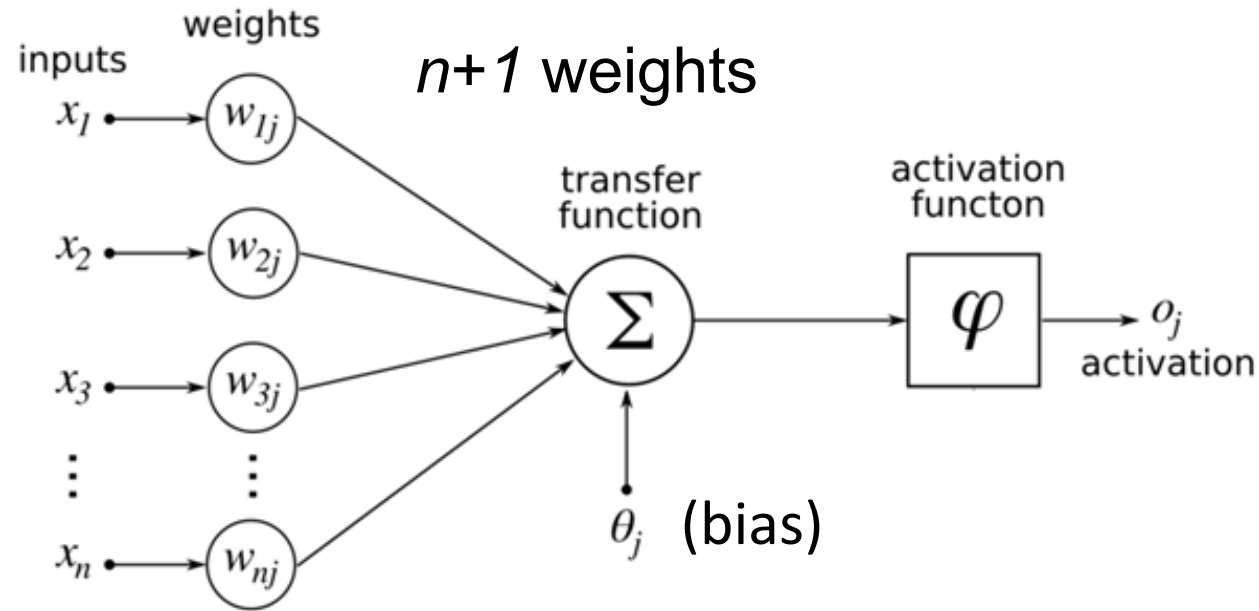
Human Neural Network



100 Billion Neurons

100 Trillion Connections

Artificial Neuron: Weights are Learned



$$o_j = \varphi \left(\theta_j + \sum_{k=0}^n x_k w_{kj} \right)$$

Activation functions φ (Nonlinear!):

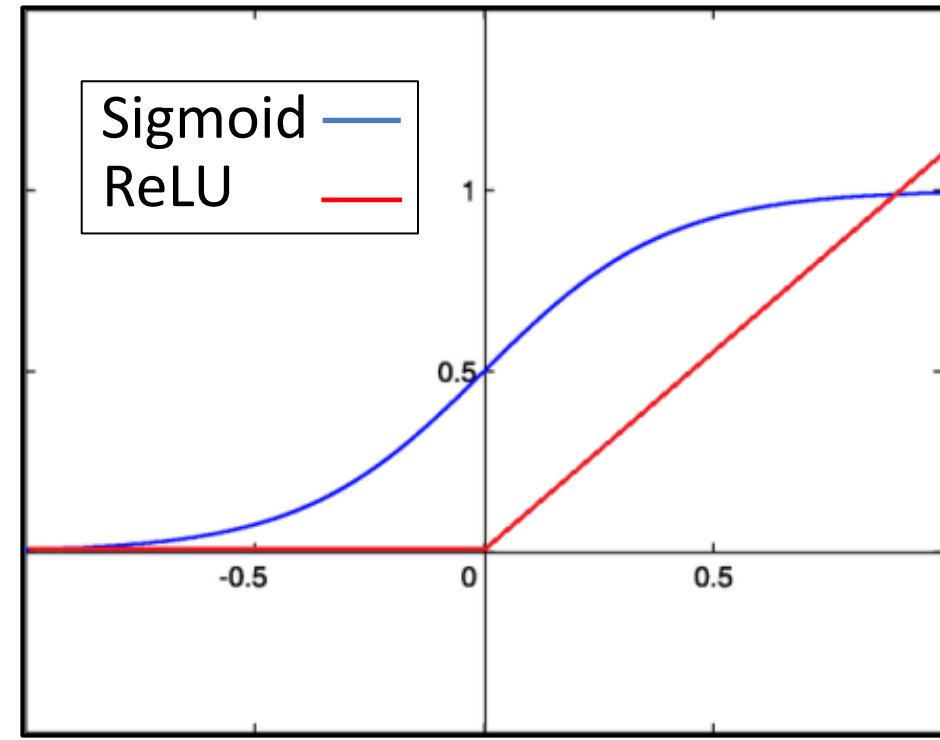
Sigmoid

$$\varphi(z) = \frac{1}{1 + e^{-z}}$$

ReLU (Rectified Linear Unit)

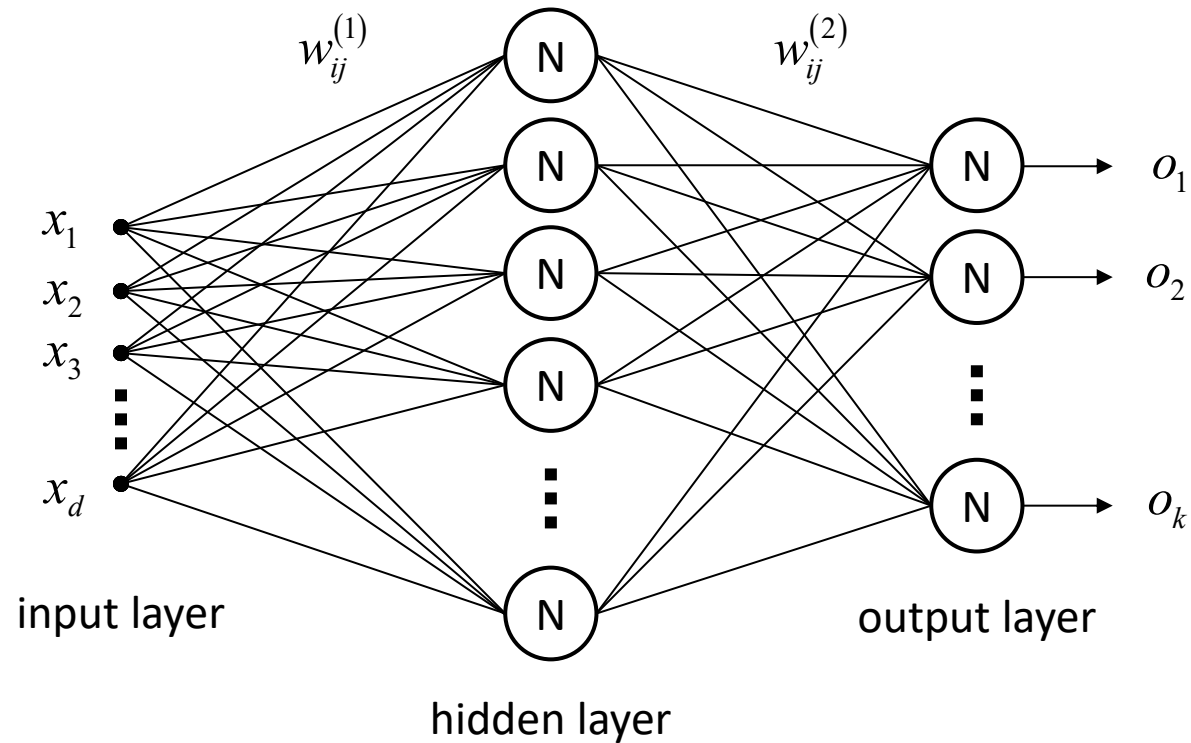
$$\varphi(z) = \max(0, z)$$

Most common in ConvNets



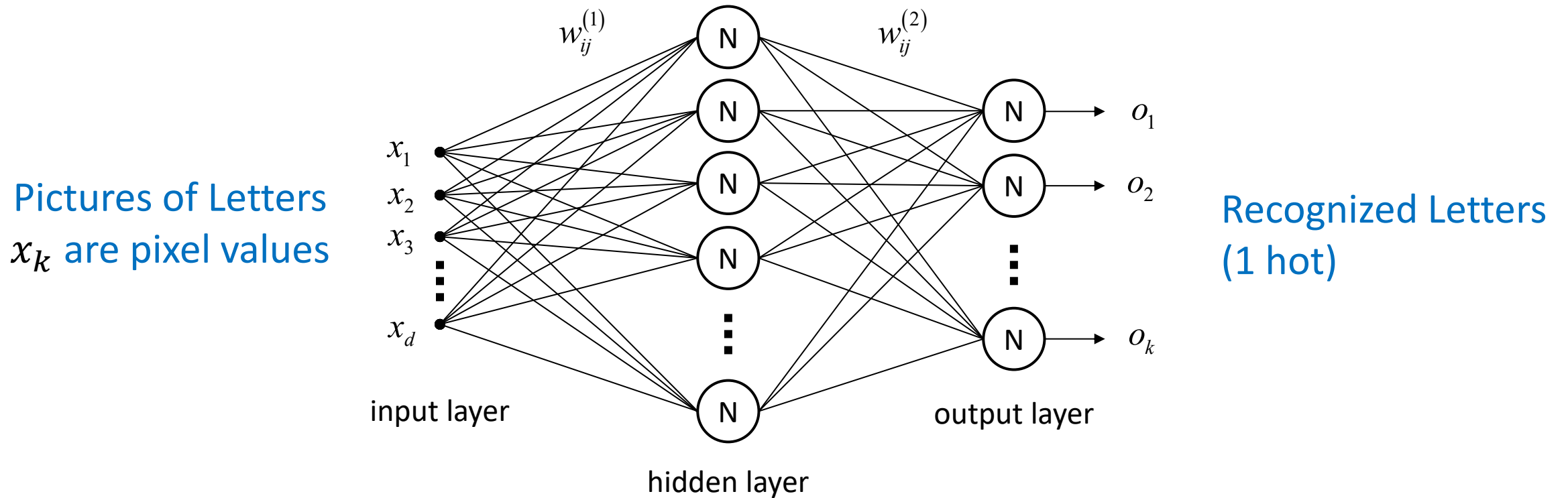
Derivatives used to find the optimal weights

Neural Network



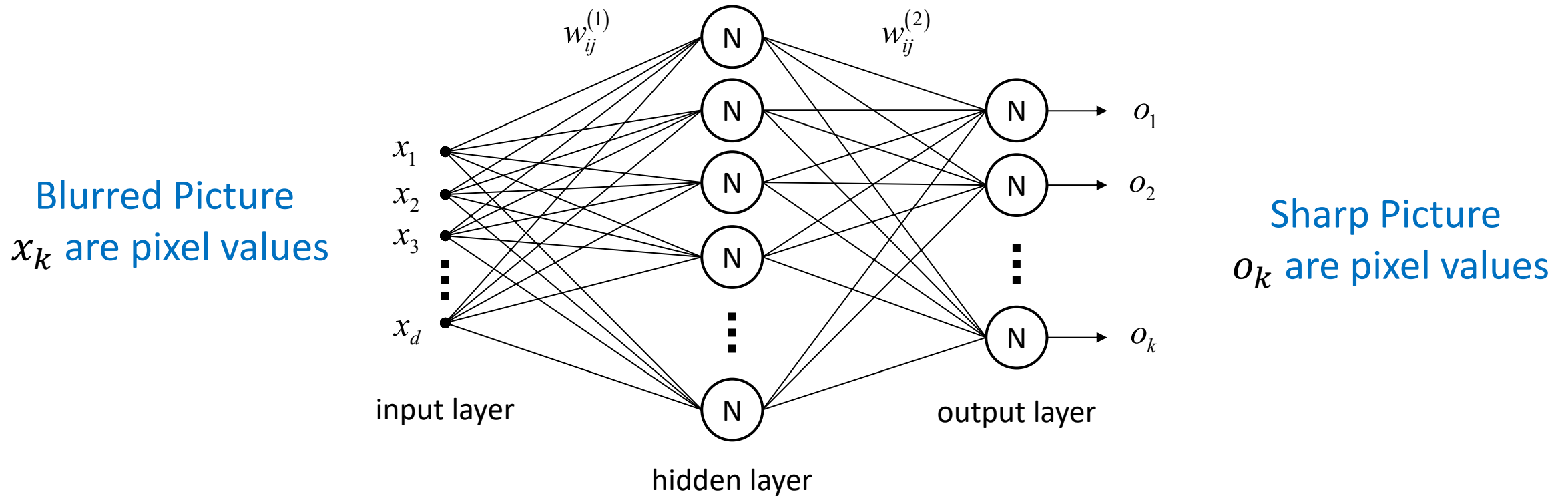
A NN, even a single hidden layer, can approximate any function from input to output (given enough neurons)

Neural Network (Classification)



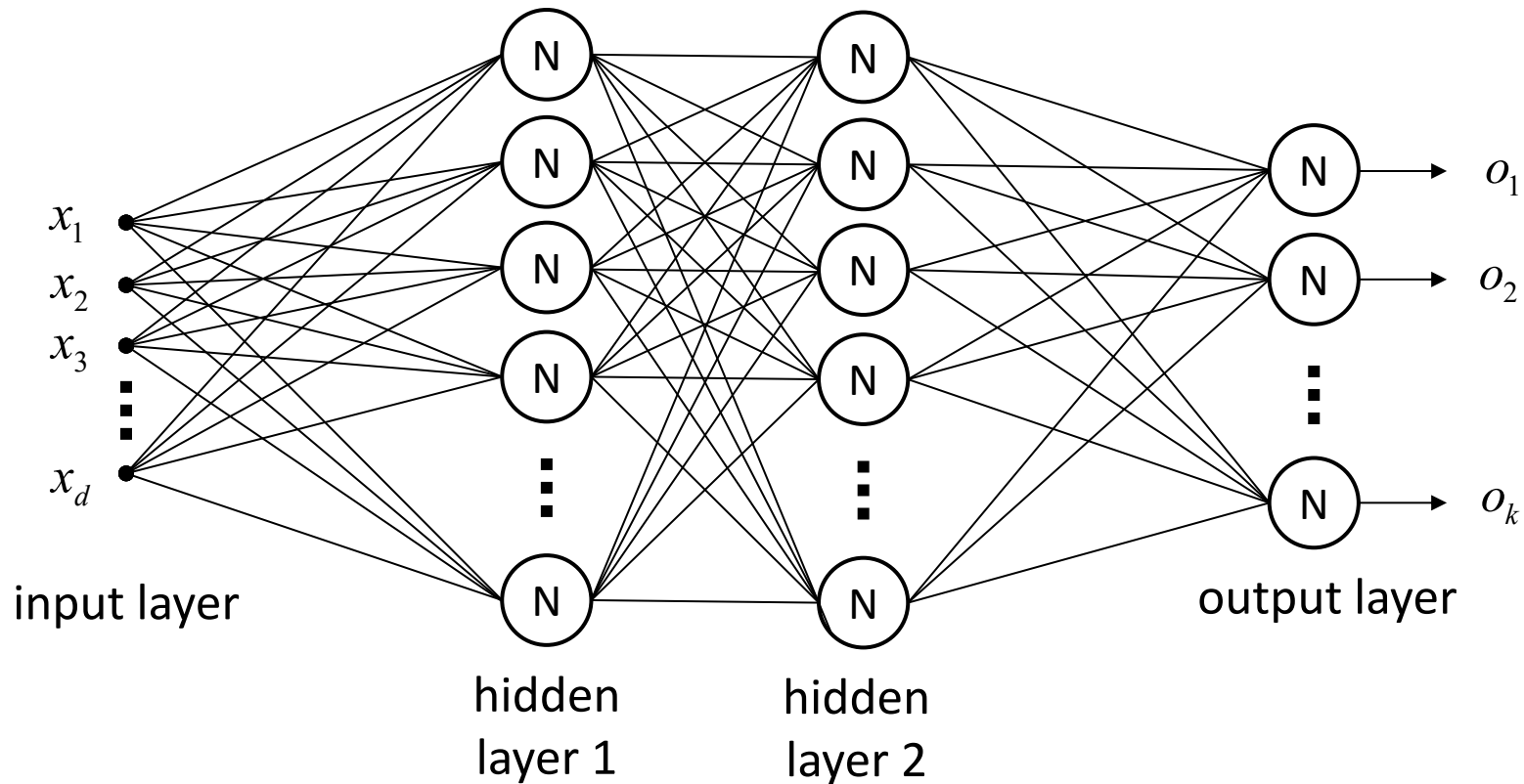
Classification: Discrete output, 1 of K classes

Neural Network (Regression)



Regression: Continuous Output (Image, Sound, Value)

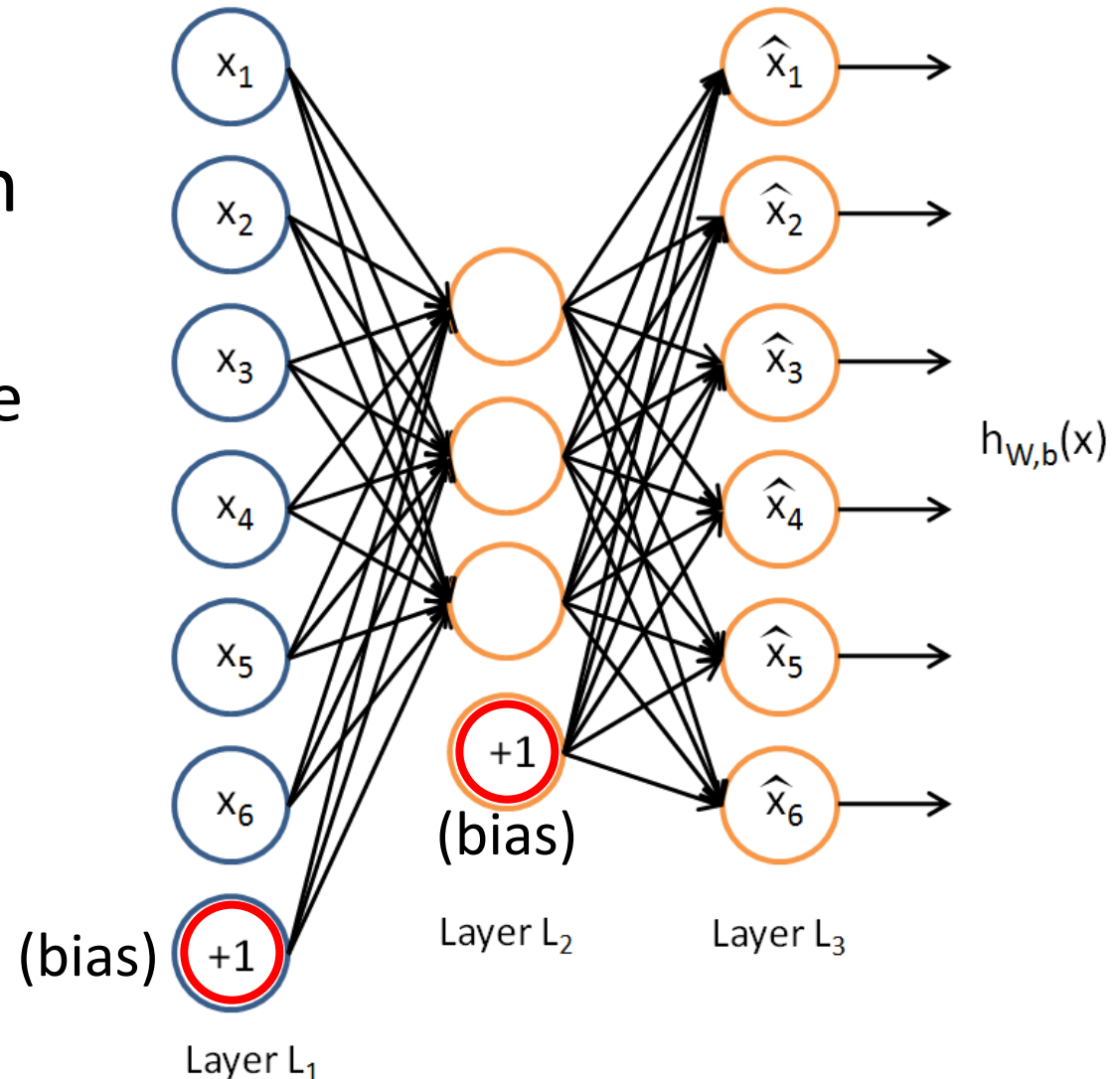
Deep Neural Network



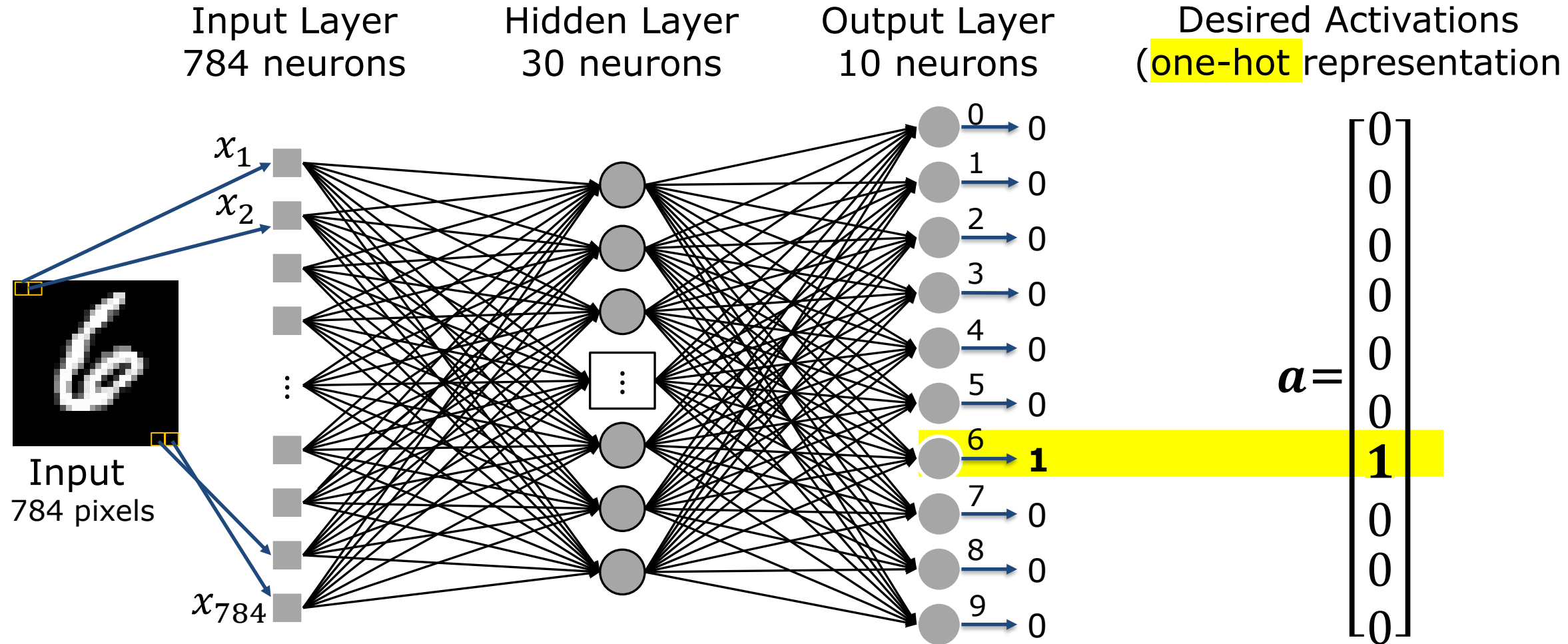
- Multiple hidden layers: 3, 20, 100, 1000, and more layers
- Layer i is Fully Connected when every node in layer i is connected to every node in layer $i-1$

Special Networks: AutoEncoders

- Output should be same as input
- One of the hidden layers has much fewer neurons than the input
 - Compact representation of the image for image comparison
 - Encoding of an image with fewer numbers for compression
- Removes noise – as random noise can not be compressed



Example: Character Recognition Neural Network



An Example Network with 95% Accuracy

Softmax for Classification

- Assume we have a classification network to determine 1 of K classes (e.g. car/person/cat)
 - Output layer has K nodes
 - Given an input we get K activations $\{z_i\}$
 - Node (class) with highest activation is selected
 - Problem: Max does not have a derivative... How to optimize?
- Softmax: Normalized any $\{z_i\}$ to “probabilities” $\{y_i\}$
 - $$y_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad y_i \geq 0; \quad \sum y_i = 1$$
- Normalization is needed to determine how far was the network from correct classification (“Loss Function”)

Loss Functions

- Measure accuracy of the results. Assume $\{y_i\}$ is correct result, and NN output is $\{z_i\}$
- Squared Pointwise Loss (Regression): $L = \sum (z_i - y_i)^2$
- Softmax Loss (Classification)
 - Assume that correct class is r : $L = -\log z_r$
 - E.g. When $z_r = 1$, no error, $L = 0$
 - When p_i is the correct probability $L = -\sum p_i \log z_i$
- Perceptual Loss: Differences in high-level features of a pretrained CNN. E.g. Levels 8-12 in VGG 19

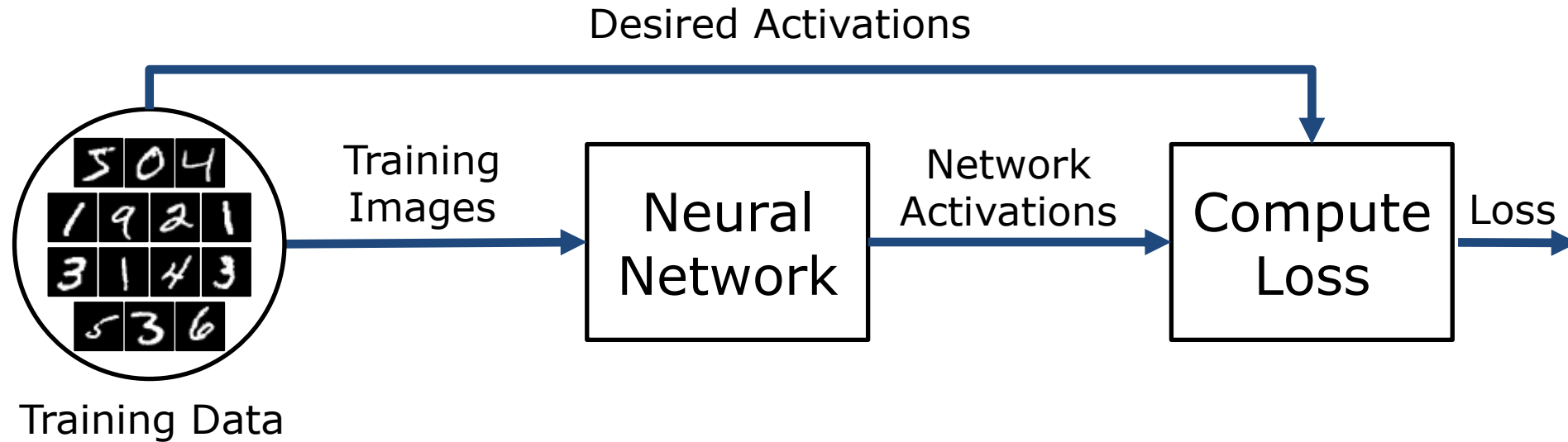
Neural Network – Training

1. Weights $\{w_{ij}^{(l)}\}$ are learned via minimization of training loss using examples in a training set (millions, billions...)
2. Each example is a pair (x, y)
 1. (x, y) : (x - Face picture, y - name of person) - Classification
 2. (x, y) : (x - Noisy image, y - clean image) - Regression
3. With best weights, for all pairs (x, y) in training set the network output $o(x)$ should be y .
4. Given current weights $\{w_{ij}^{(l)}\}$ we compare the network output $o(x)$ to y , obtaining training loss $L_S(w_{ij}^{(l)})$

Neural Network – Training

- Given the training set $S = \{(x^m, y^m)\}_{m=1}^M$ we try to compute the weights $\{w_{ij}^{(l)}\}$ that minimize the training loss $L_S(w_{ij}^{(l)})$ over all training examples
- This is very hard minimization. Typically solved with Stochastic Gradient Descent (using derivatives of L for each weight w_{ij})
 - A mini-batch of examples is randomly selected
 - Weights minimizing loss of this mini-batch are computed using back-propagation
 - Repeated many times with different mini-batches
- Computing derivatives of loss for each weight using chain rule

Training Process



1. Initialize weights and biases of network (e.g. random values)
2. Randomly select training images in a mini-batch
3. Compute Loss for the all images in mini batch
4. Update weights and biases by back-propagation using derivatives of loss function by each weight (Chain rule)
5. Repeat from (2). (Epoch – When we used once all training data)

What is Neural Networks Doing?

- Learning a function from examples
- Problem 1: Network is not rich enough to describe the function.
- Problem 2: Network can fit perfectly all input examples, but fail for other cases (overfitting)
- Challenge: Design a network powerful enough to solve the problem and small enough to avoid overfitting
- Approach: experience; trial and error...

Learning from Data

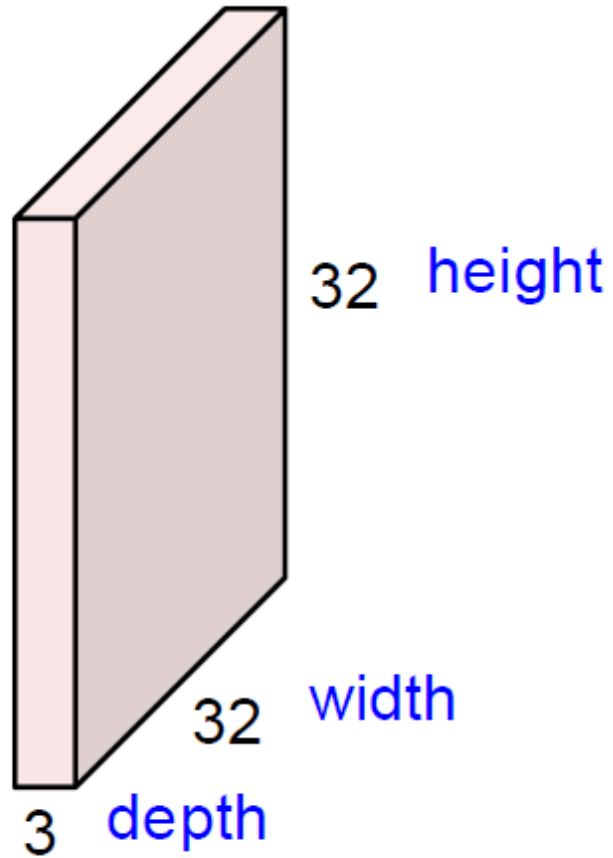
- A given data set (examples) is divided to training data and test data (E.g. ratio 8:2 .. 100:1)
- Train the network using the training data. Training error should go down during training.
- Test the network on the test data (not used for training!). Watch the error on test data.
- Problem 1: Training error does not go down.
 - Reasons: network too small, wrong loss function, etc...
- Problem 2: Training error goes down; test error goes up
 - Reasons: overfitting.

Convolutional Neural Networks (CNN)

- More limited than fully connected networks: some layers use only a convolution (less weights)
 - e.g. 25 weights per layer instead of 1,000,000.
- Used when convolutions are meaningful: images, signals (sound)
- Reduces complexity of Neural Networks with the reduced number of weights
- Most CNNs have many convolution layers followed by some fully connected layers

Convolutional Layer

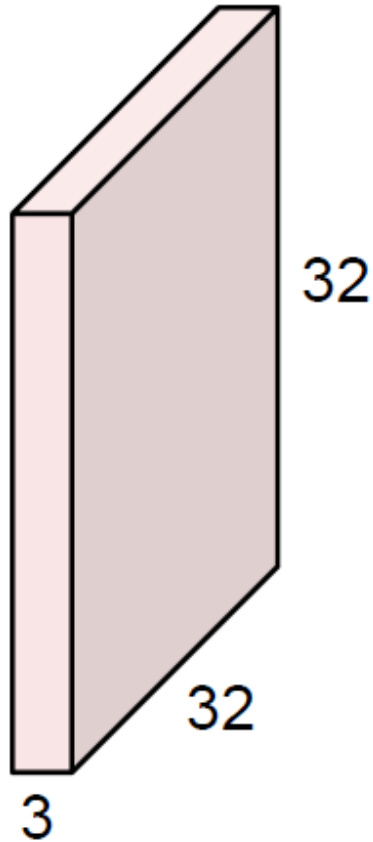
32x32x3 image



A color input image has 3 channels for (R, G, B)

Convolutional Layer

32x32x3 image



5x5x3 filter

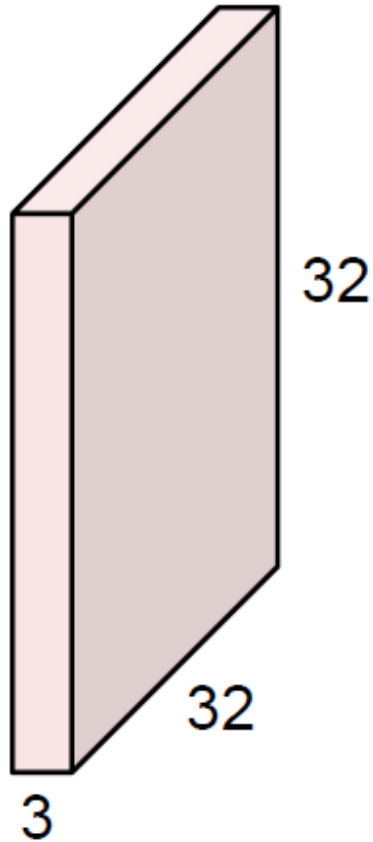


Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

A color input image has 3 channels for (R, G, B)

Convolutional Layer

32x32x3 image



5x5x3 filter

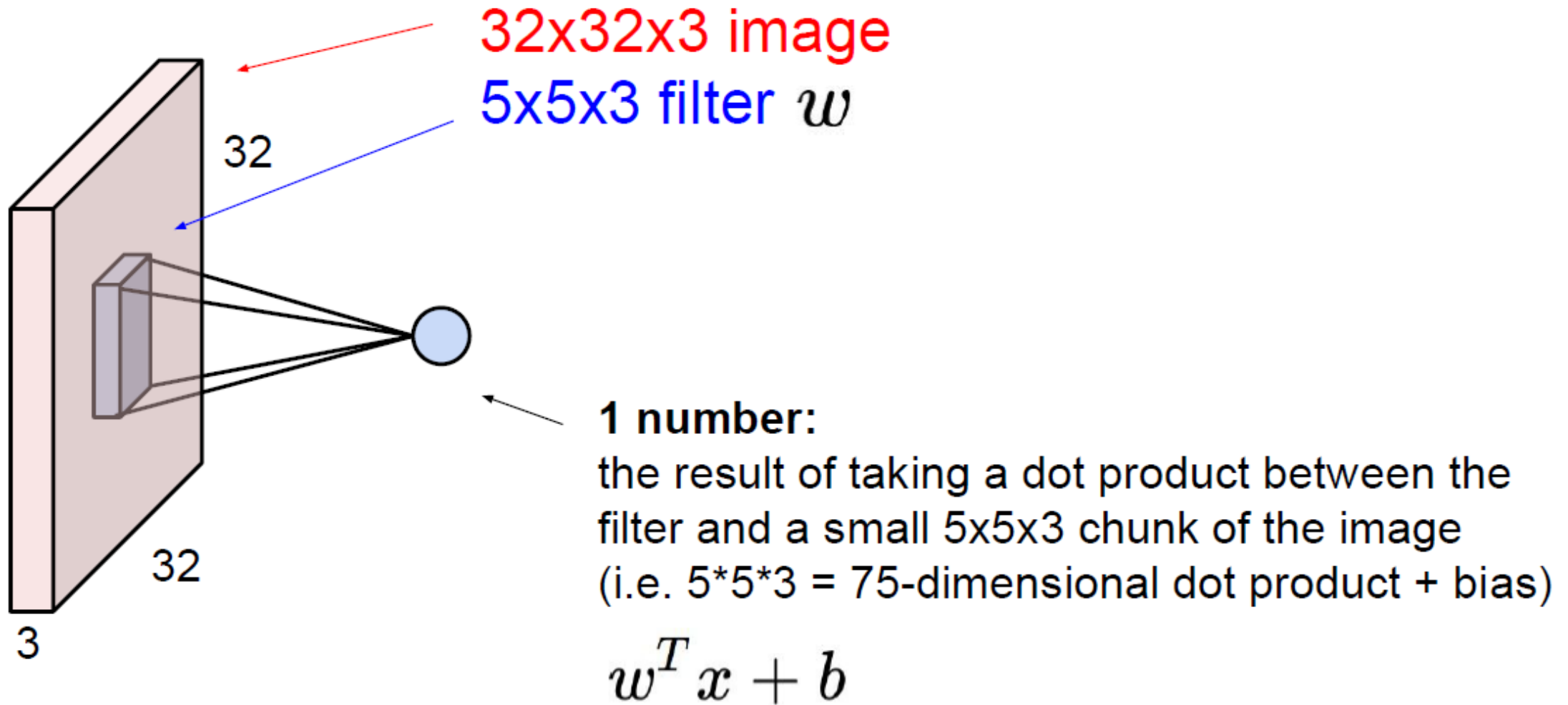


Filters always extend the full depth of the input volume

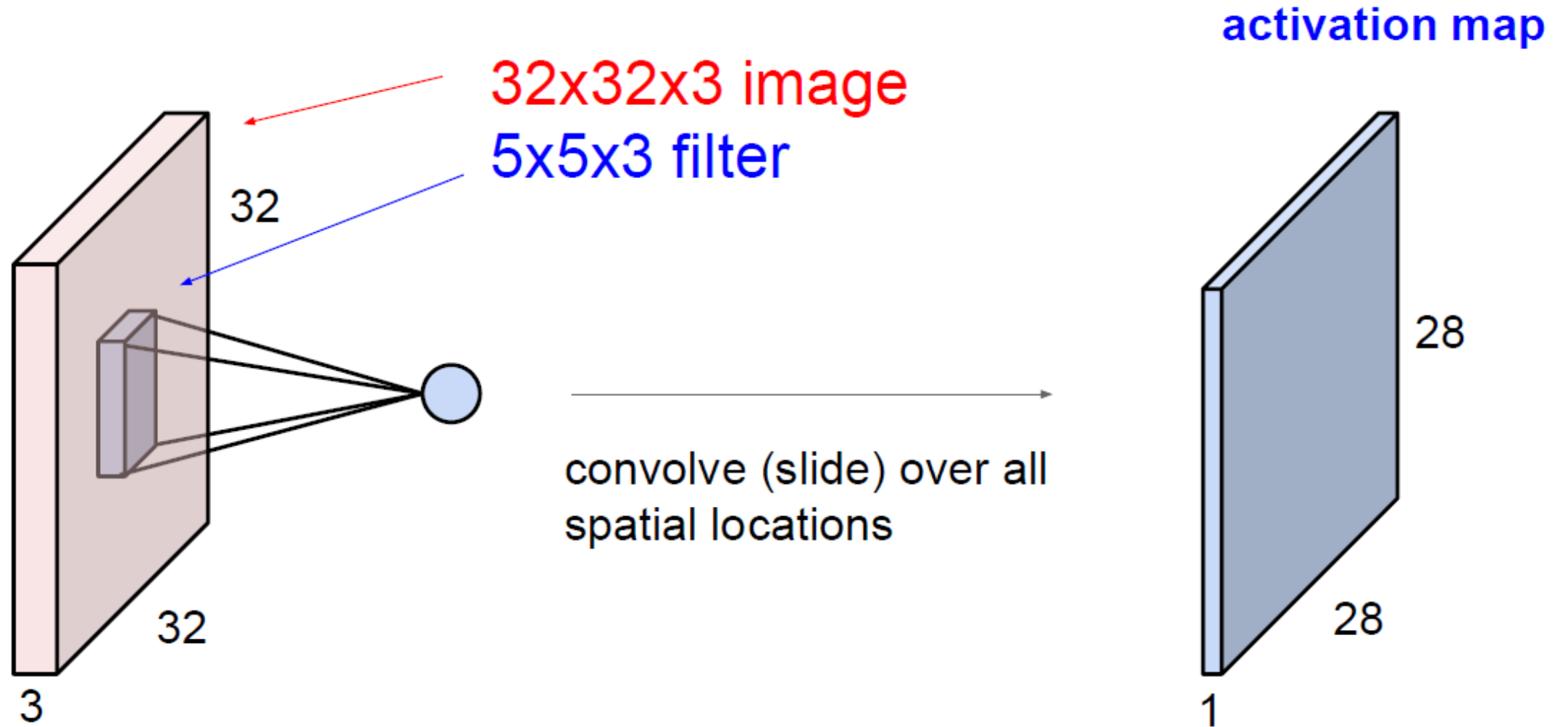
Convolve the filter with the image
i.e. “slide over the image spatially,
computing dot products”

A color input image has 3 channels for (R, G, B)

Convolutional Layer

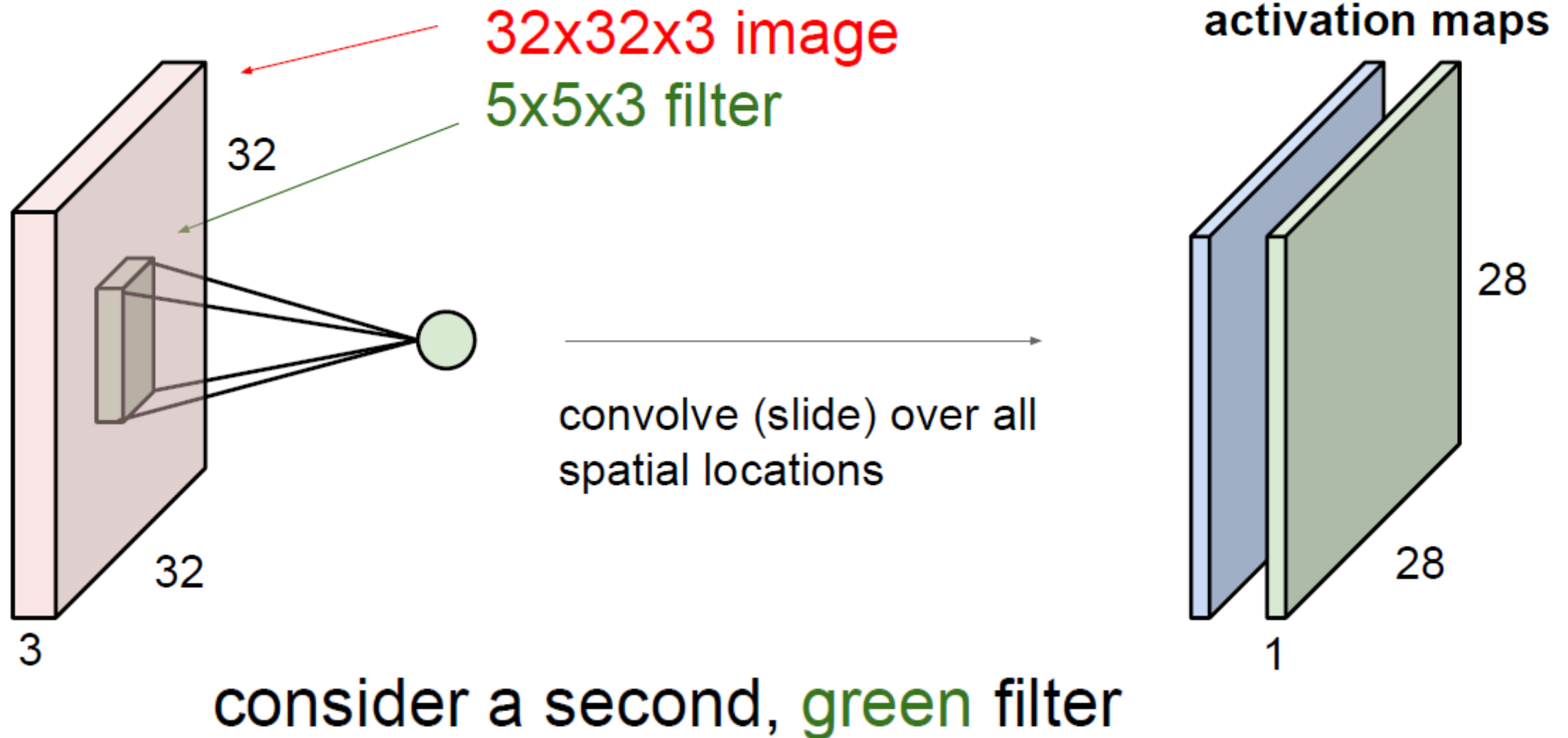


Convolutional Layer



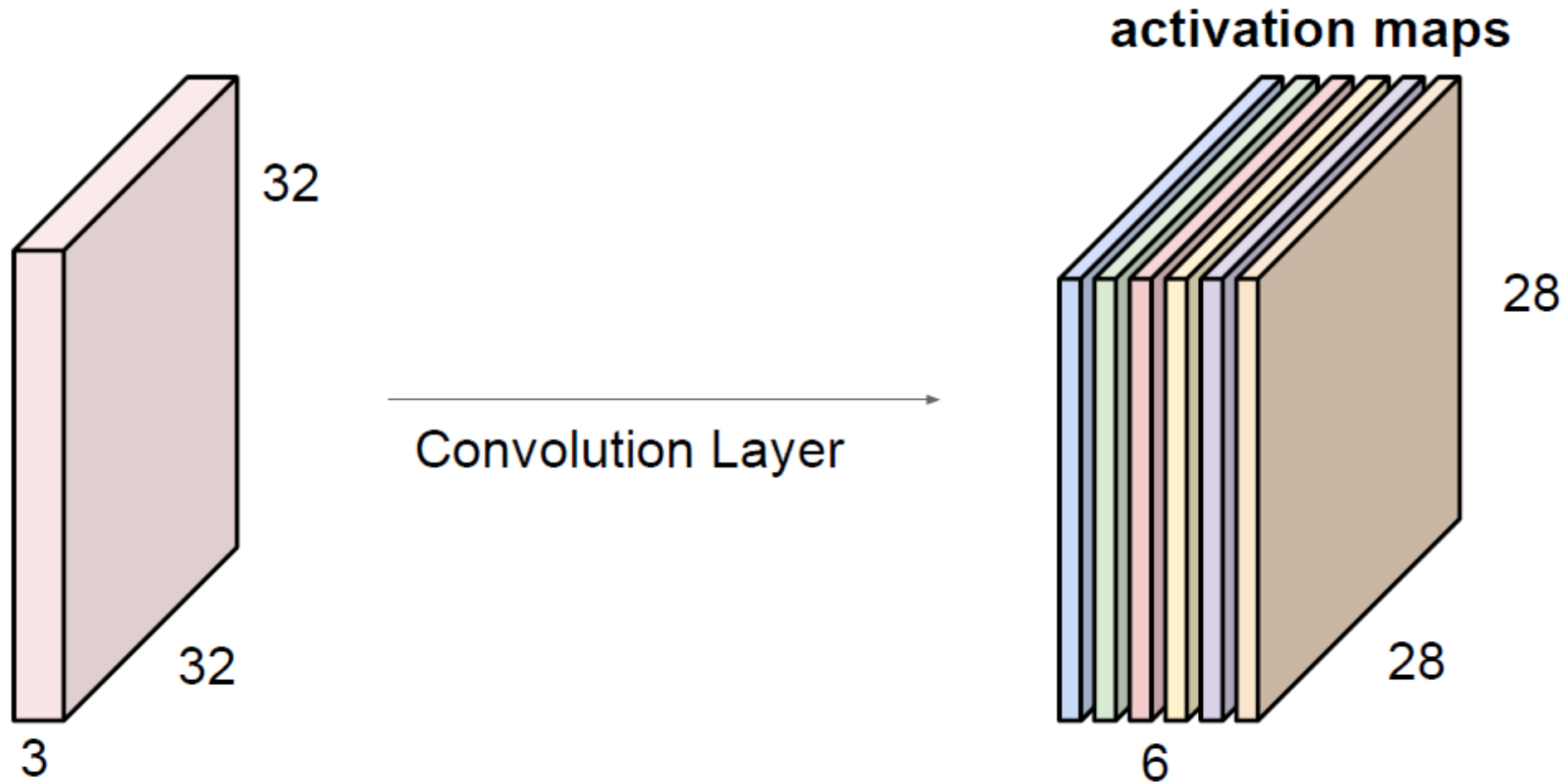
If convolution does not extend beyond image, 32 by 32 becomes 28 by 28

Convolutional Layer



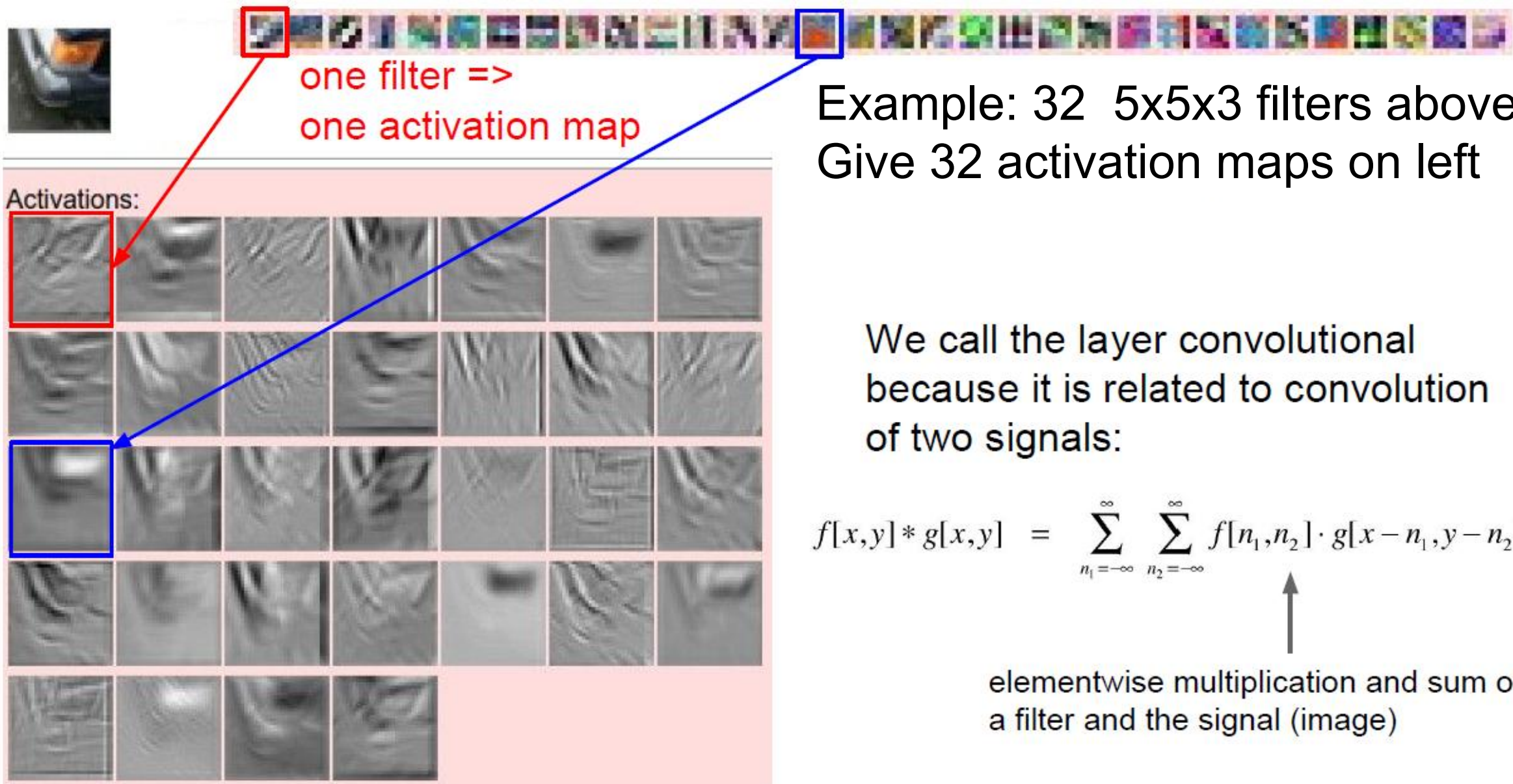
Convolutional Layer

For example, if we had 6 5x5 filters, we'll get 6 separate activation maps:



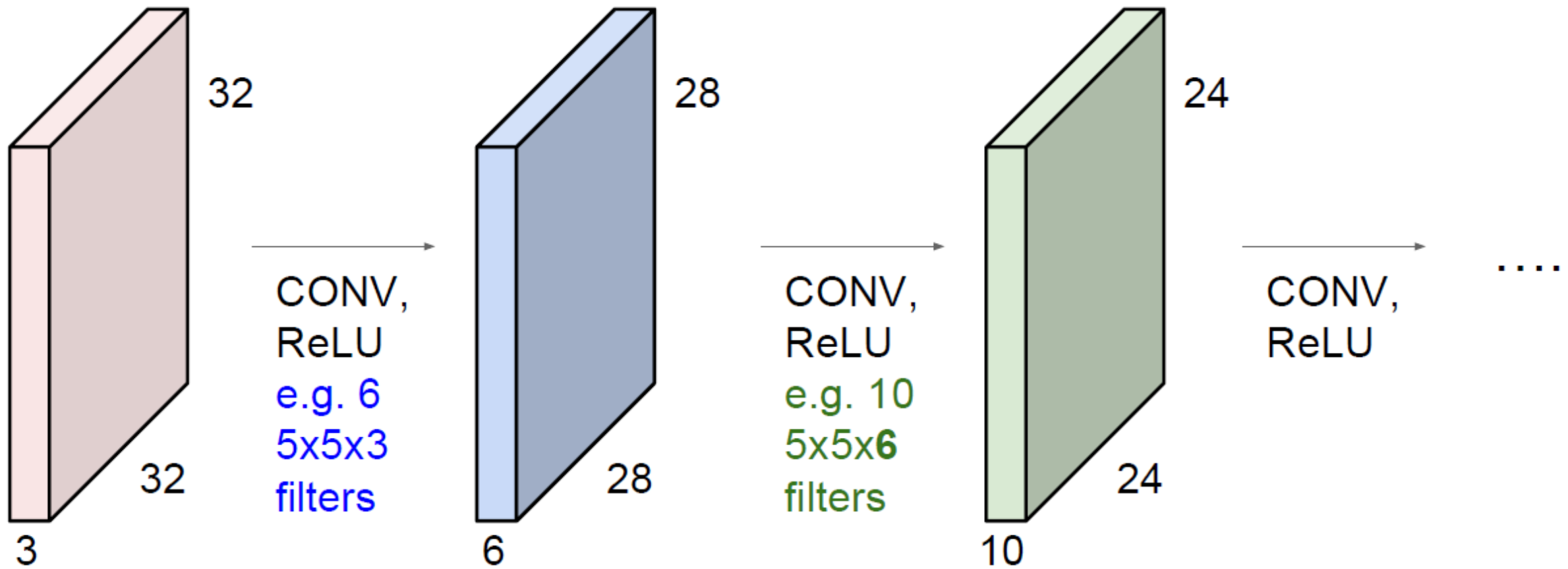
We stack these up to get a “new image” of size 28x28x6!

Sample Activations of a Convolutional Layer

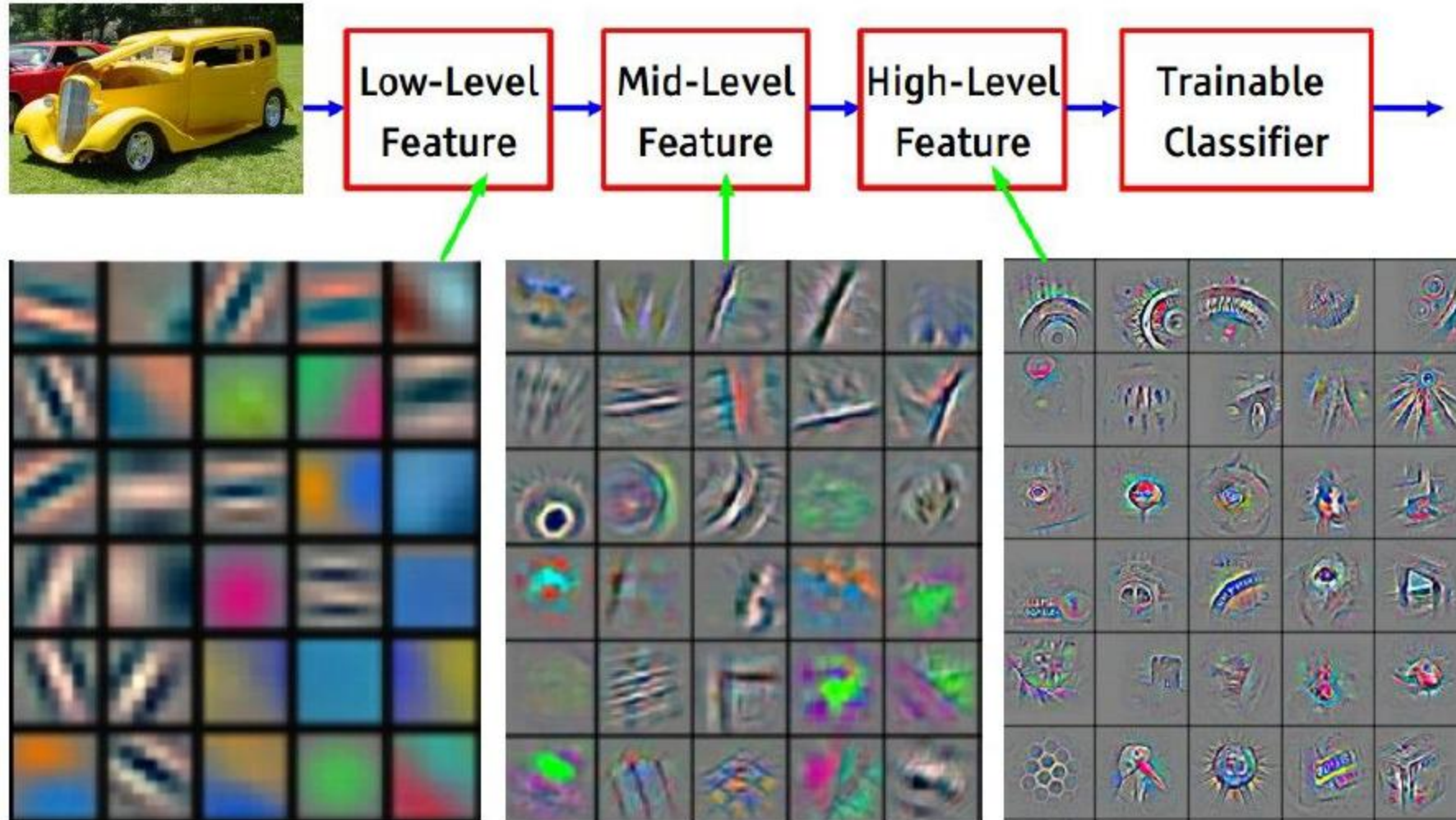


Convolutional Network

Preview: ConvNet is a sequence of Convolutional Layers, interspersed with activation functions



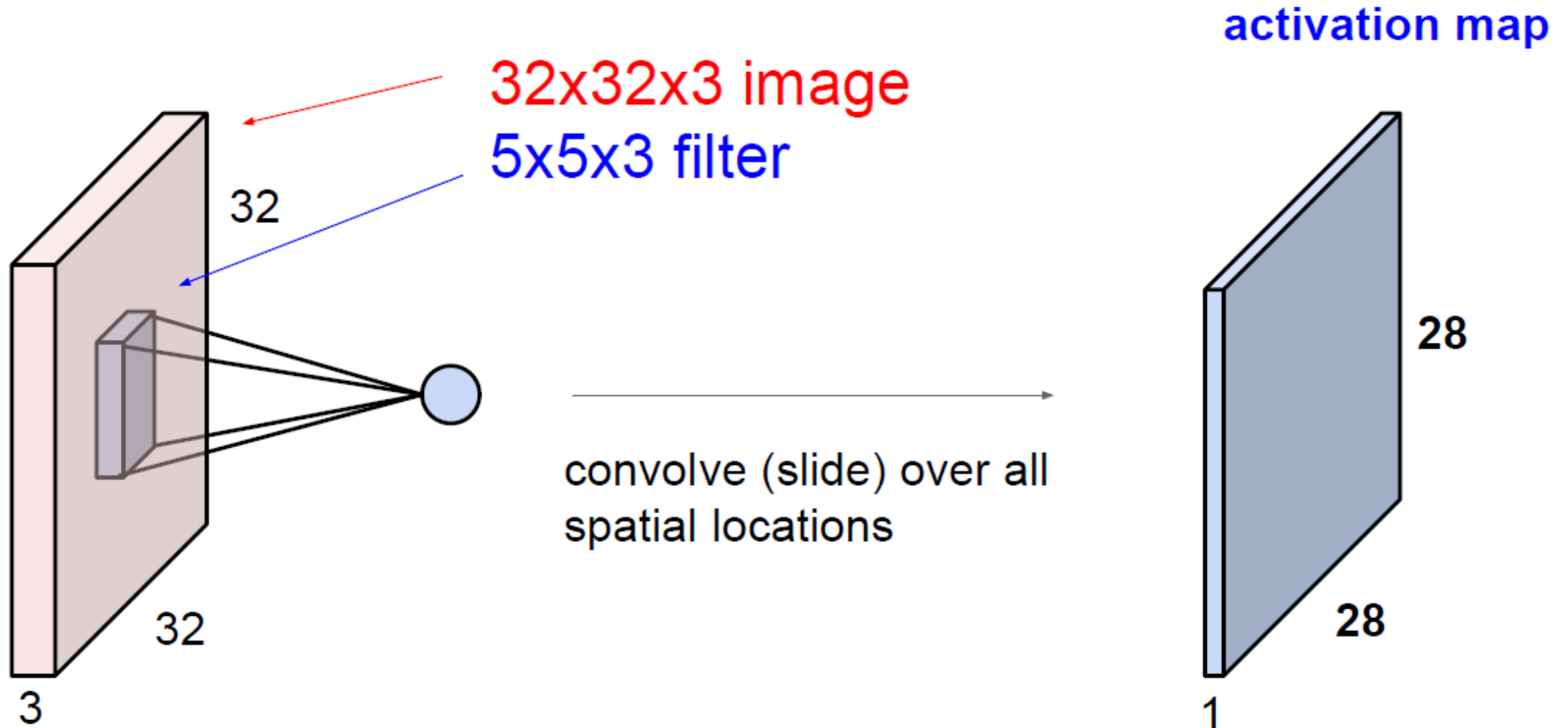
Convolutional Layer



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus 2013]

Convolutional Layer

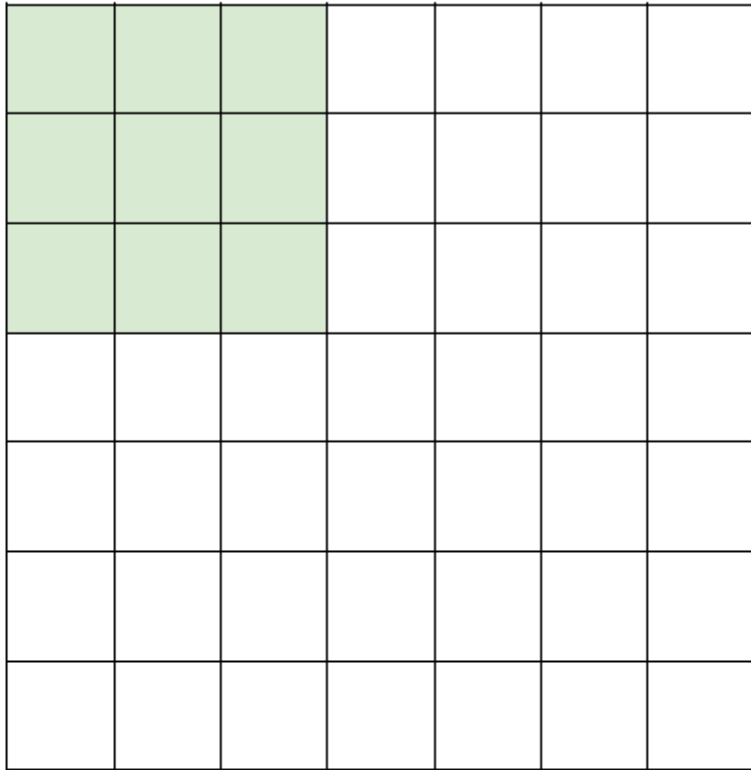
A closer look at spatial dimensions:



Convolutional Layer

A closer look at spatial dimensions:

7



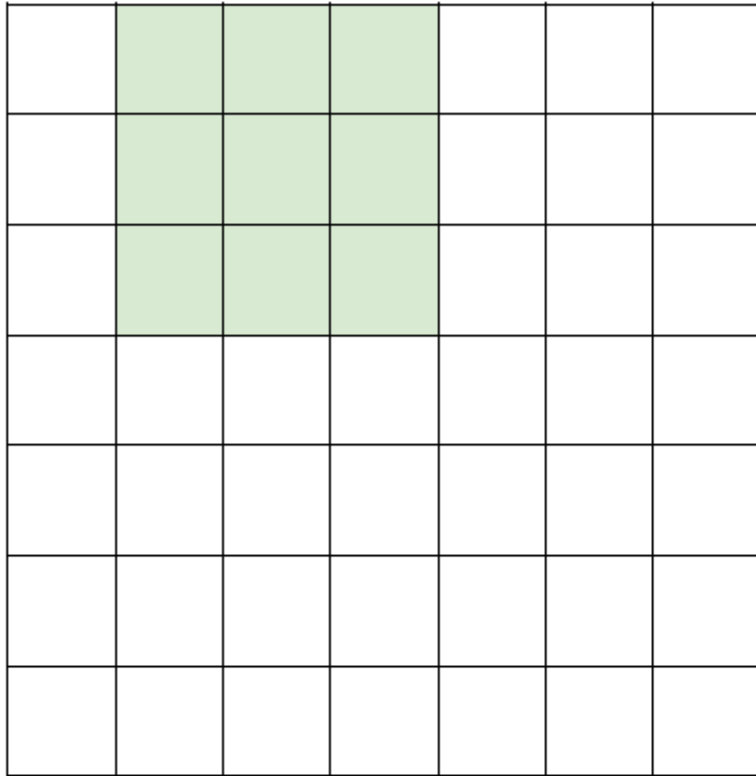
7x7 input (spatially)
assume 3x3 filter

7

Convolutional Layer

A closer look at spatial dimensions:

7



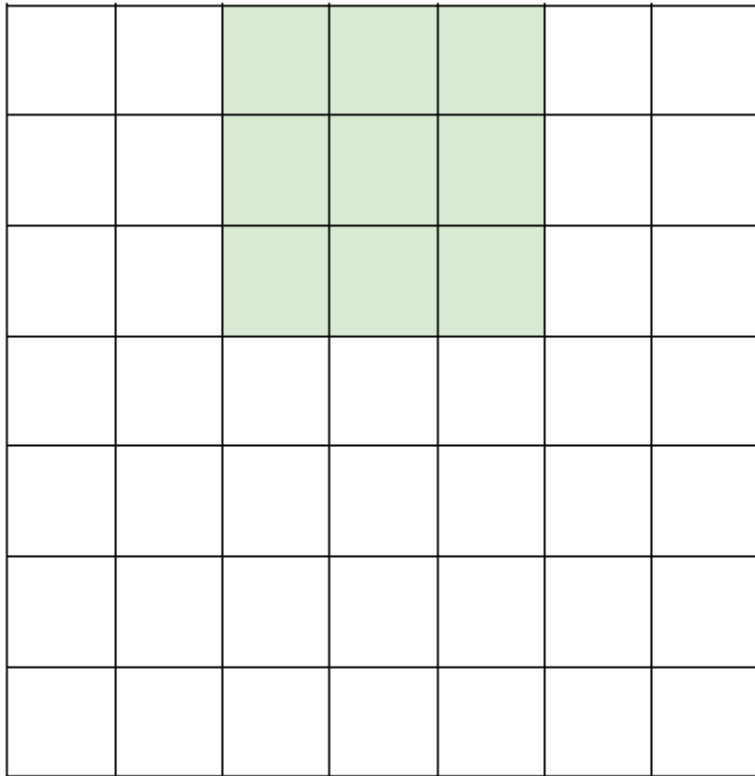
7x7 input (spatially)
assume 3x3 filter

7

Convolutional Layer

A closer look at spatial dimensions:

7



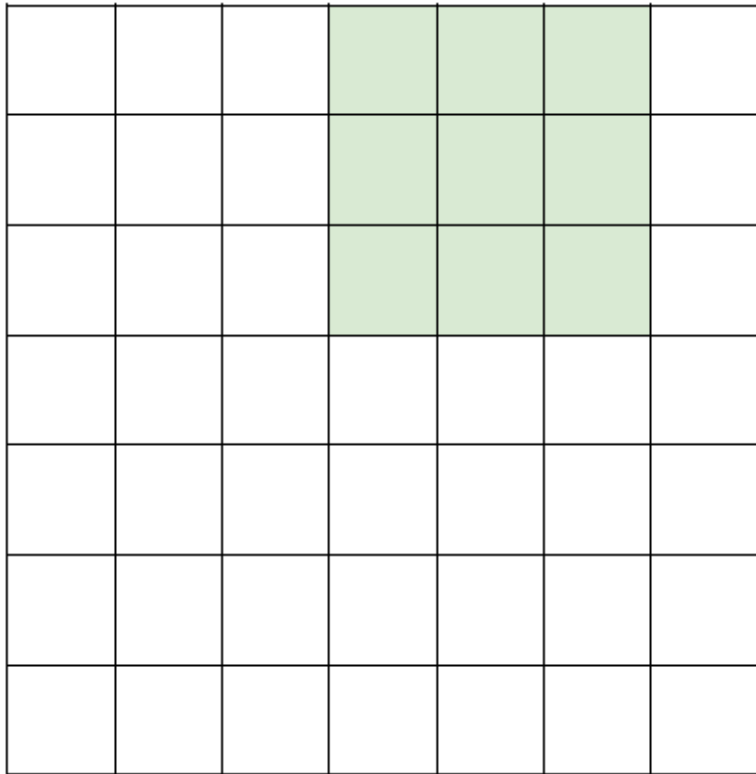
7x7 input (spatially)
assume 3x3 filter

7

Convolutional Layer

A closer look at spatial dimensions:

7



7

7x7 input (spatially)
assume 3x3 filter

Convolutional Layer

A closer look at spatial dimensions:

7

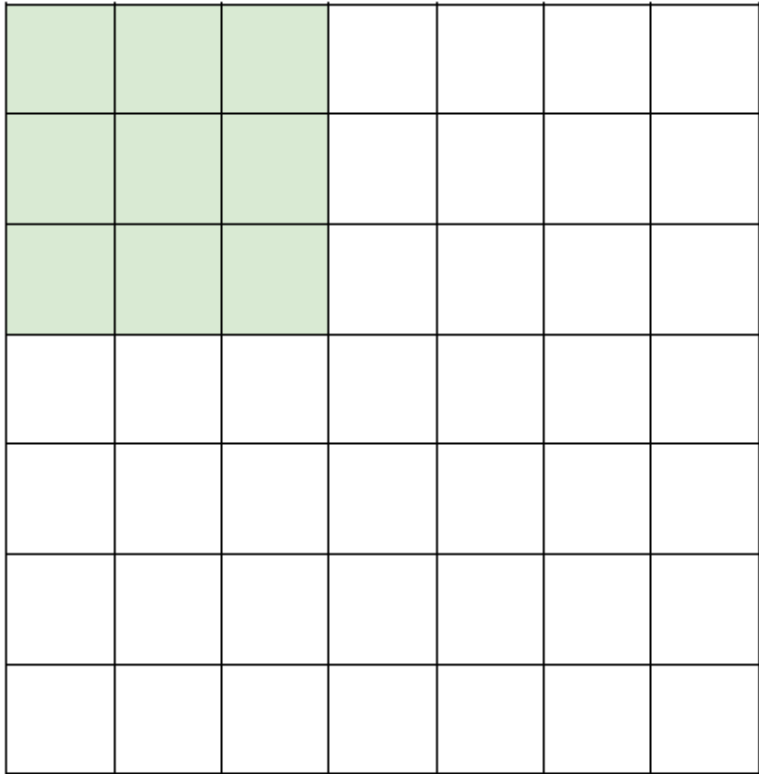
7

7x7 input (spatially)
assume 3x3 filter
=> **5x5 output**

Convolutional Layer

A closer look at spatial dimensions:

7



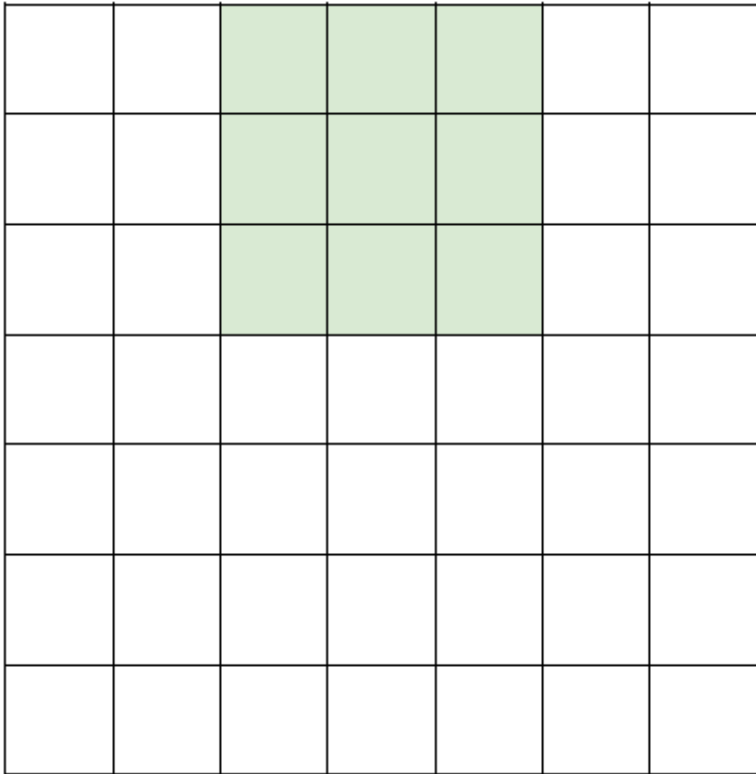
7

7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolutional Layer

A closer look at spatial dimensions:

7



7

7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**

Convolutional Layer

A closer look at spatial dimensions:

7

7

7x7 input (spatially)
assume 3x3 filter
applied **with stride 2**
=> 3x3 output!

Convolutional Layer

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

(recall:)

$$(N - F) / \text{stride} + 1$$

Convolutional Layer

In practice: Common to zero pad the border

0	0	0	0	0	0			
0								
0								
0								
0								

e.g. input 7x7

3x3 filter, applied with **stride 1**

pad with 1 pixel border => what is the output?

7x7 output!

$$(N - F) / \text{stride} + 1$$

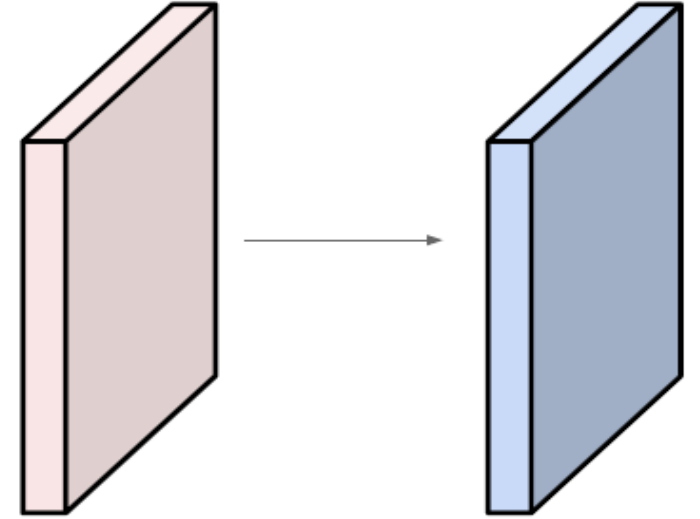
Convolutional Layer

Examples time:

Input volume: **32x32x3**

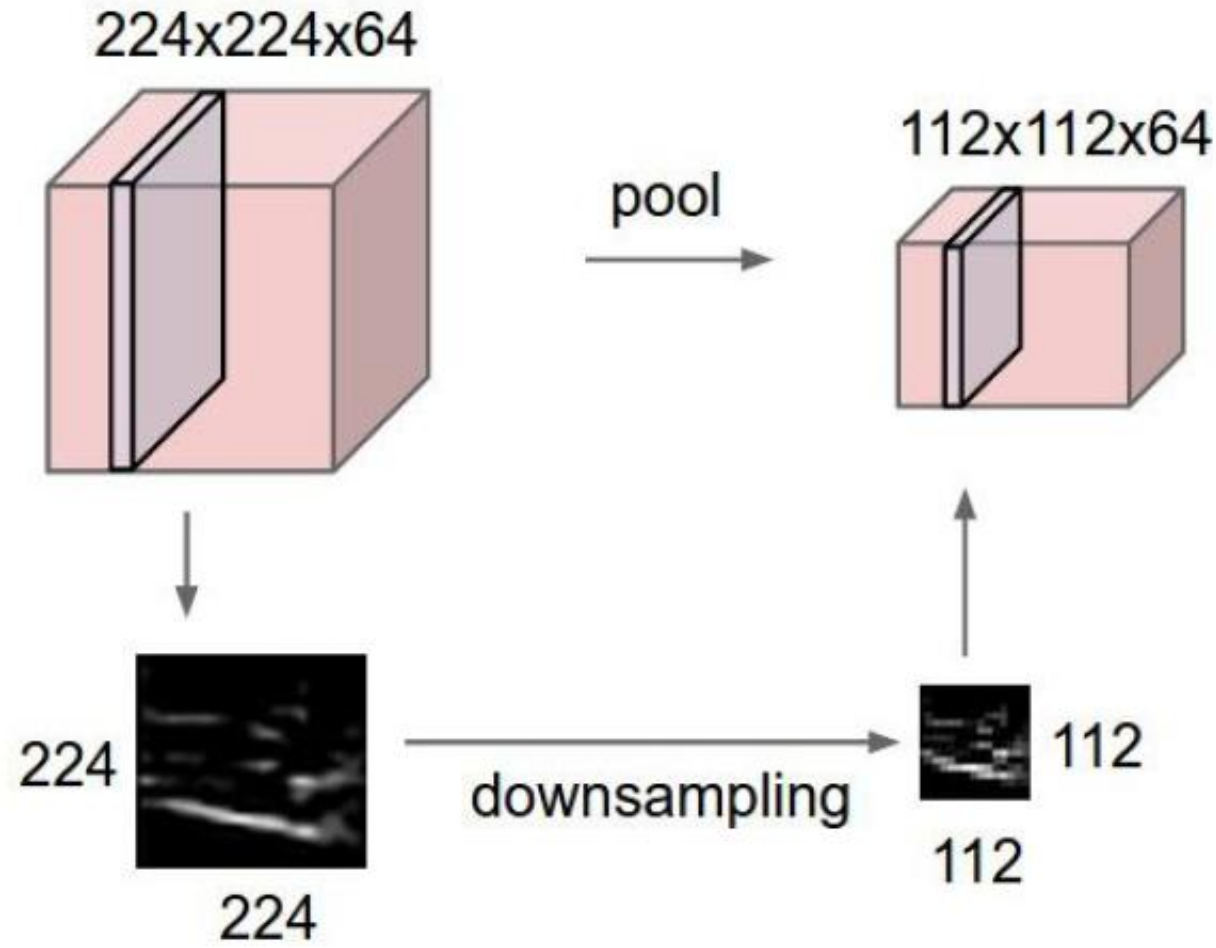
10 **5x5** filters with stride 1, pad 2

Number of parameters in this layer?



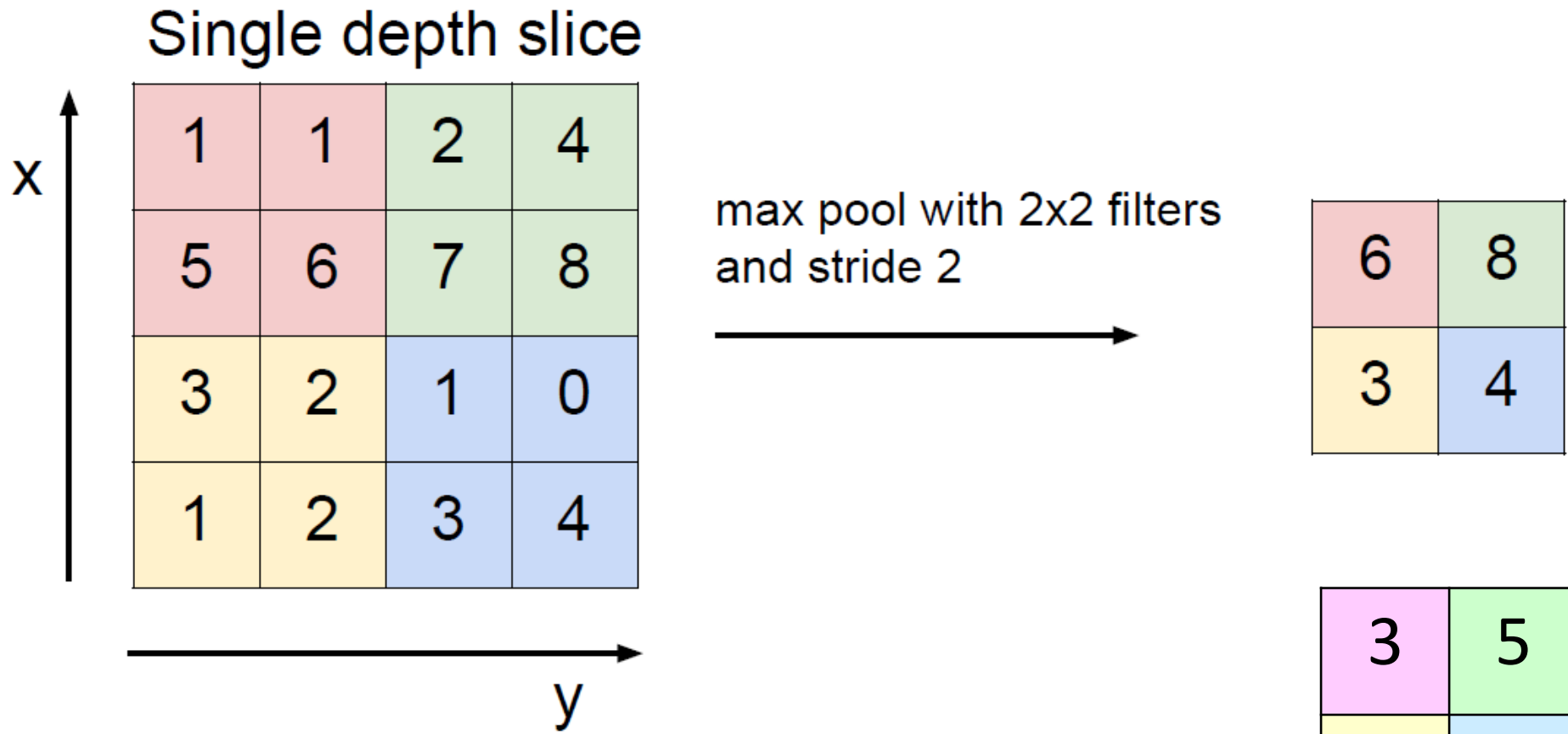
Pooling Layer

- Makes the representations smaller and more manageable
- Operates over each activation map (Channel) independently



Pooling Layer

Common choice – max pooling (Nonlinear):

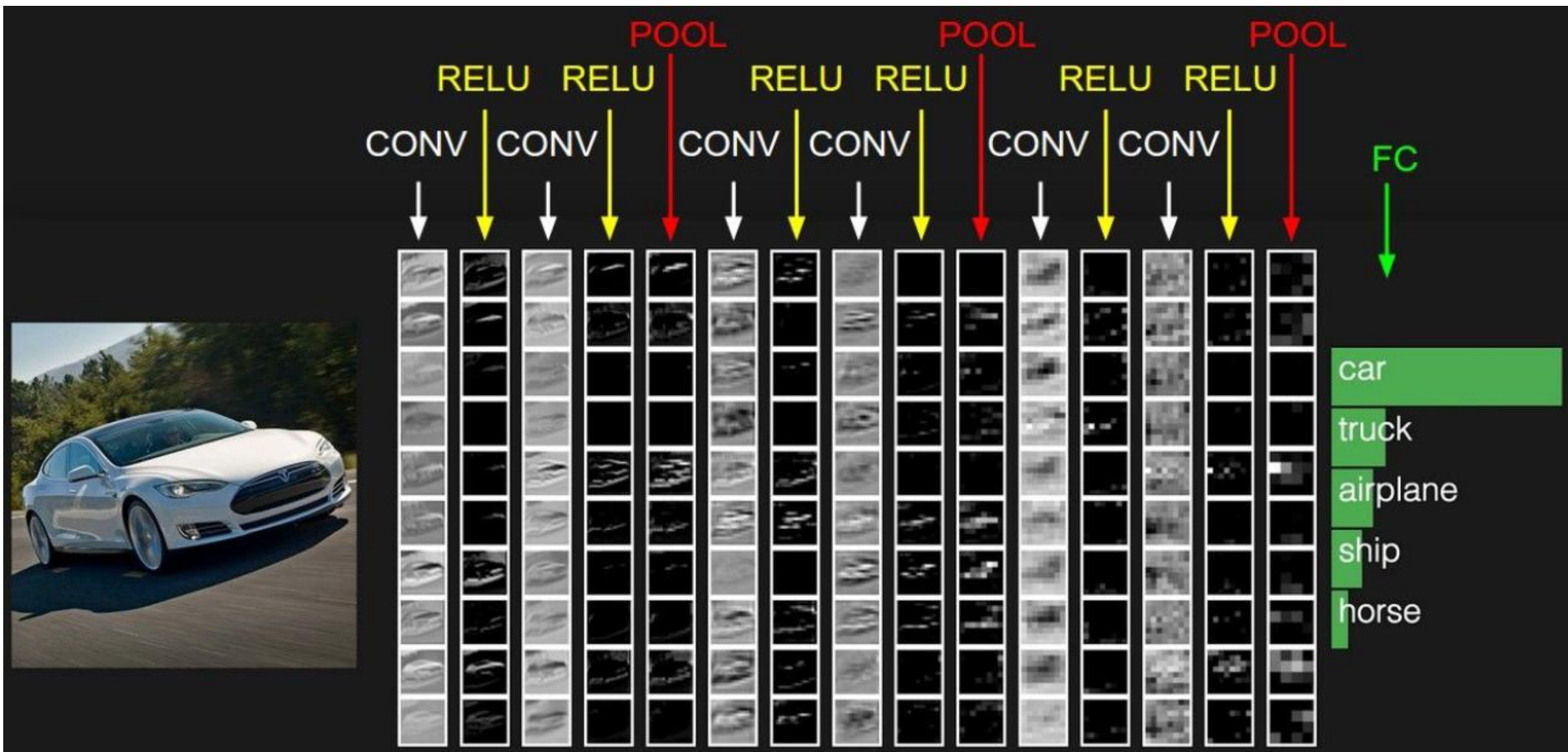


Another possibility - average pooling:

We used Convolution & Pooling Before

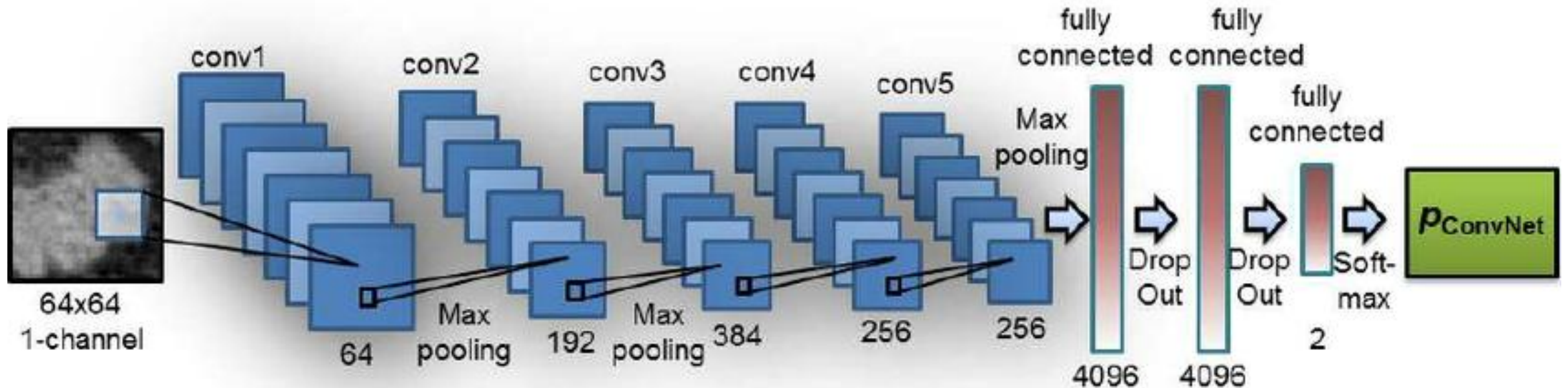
- Gaussian & Laplacian Pyramids
- Fixed weights (“hand crafted”, not trained)
- Average Pooling, or stride=2 & No pooling
- The major power of neural network:
 - Weights are learned by training from many examples.
No need to guess “hand crafted” features.
 - Features and classifiers are trained together.
 - Non-Linearity (e.g. ReLU, Max pooling)

Classic ConvNet



Fully Connected (Dense) Layer

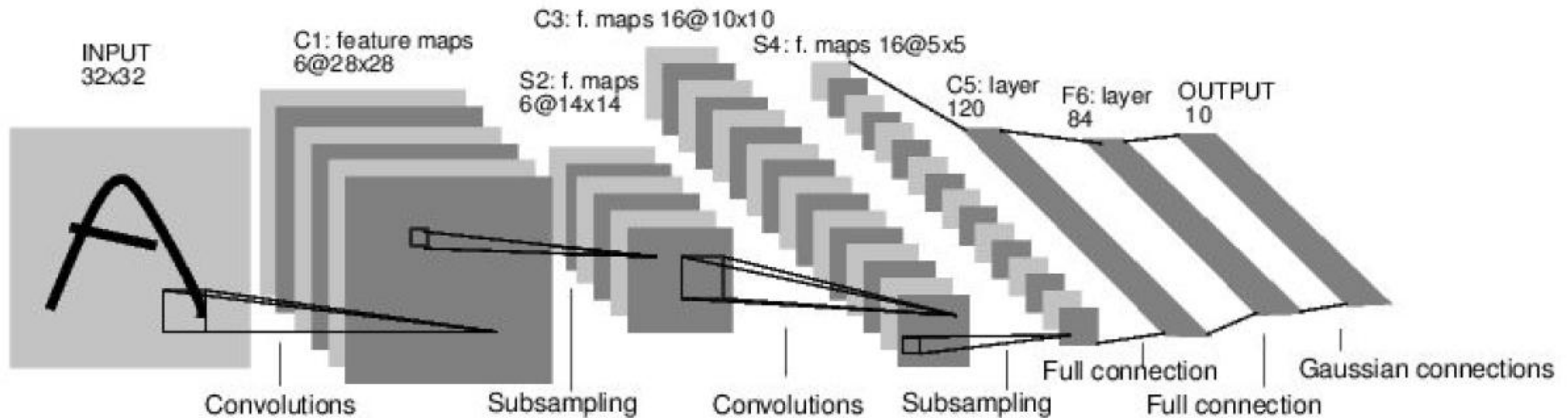
- Contains neurons that connect to the entire input volume, as in ordinary neural networks



Case Studies

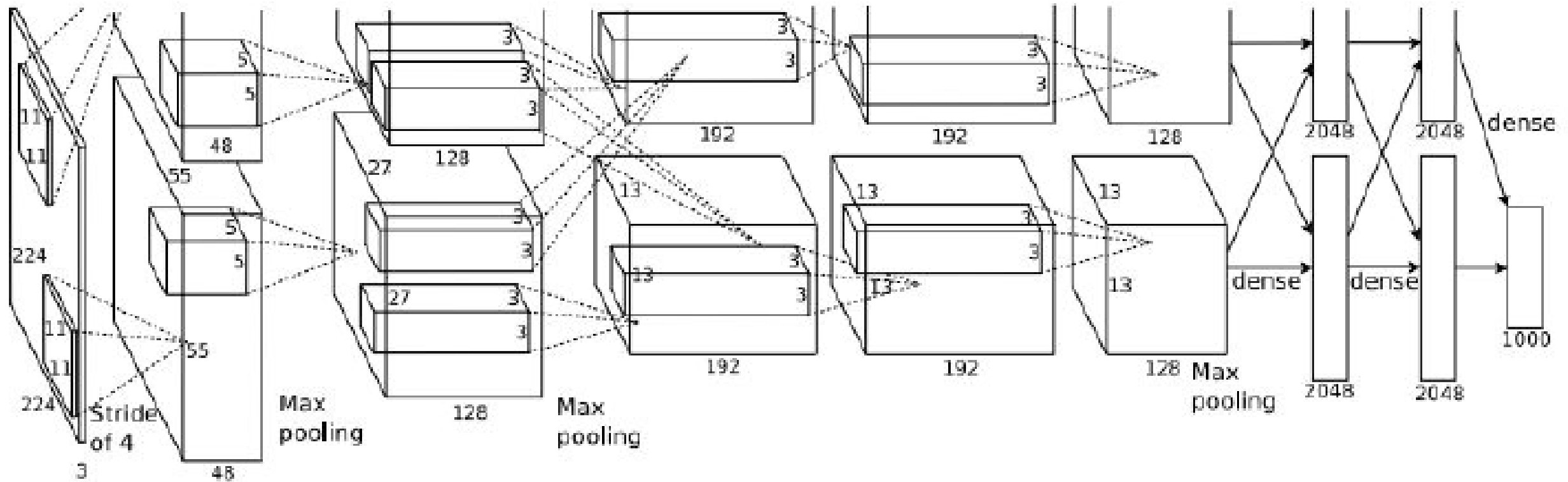
LeNet-5 [LeCun et al. 1998]

[LeCun et al.,



Conv filters were 5x5, applied at stride 1
Subsampling (Pooling) layers were 2x2 applied at stride 2
i.e. architecture is [CONV-POOL-CONV-POOL-CONV-FC]

AlexNet [Krizhevsky et al. 2012]



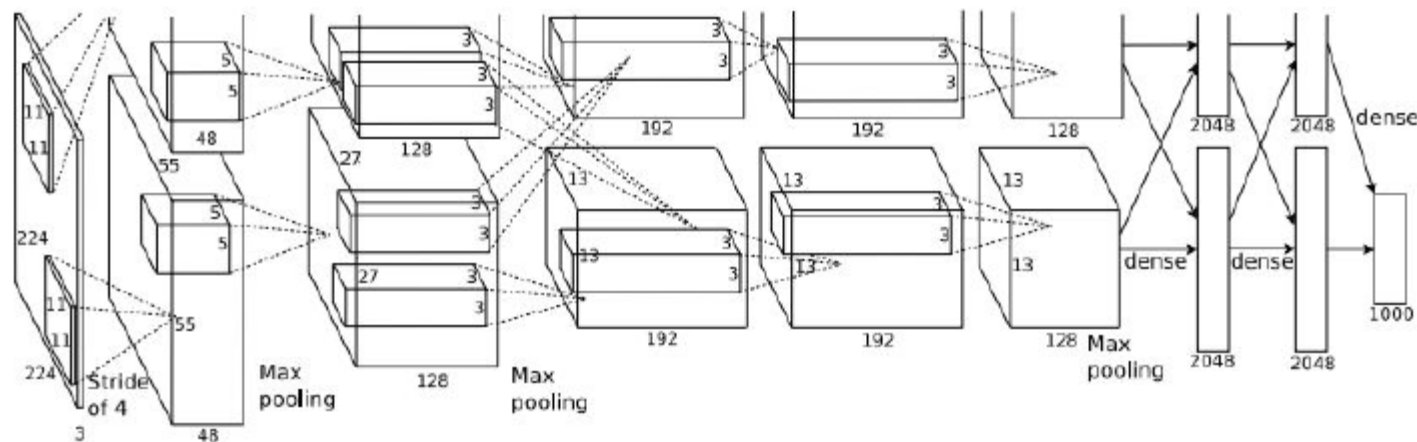
Input: 227x227x3 images

After CONV1: 55x55x96

After POOL1: 27x27x96

...

AlexNet [Krizhevsky et al. 2012]



Full (simplified) AlexNet architecture:

[227x227x3] INPUT

[55x55x96] CONV1: 96 11x11 filters at stride 4, pad 0

[27x27x96] MAX POOL1: 3x3 filters at stride 2

[27x27x96] NORM1: Normalization layer

[27x27x256] CONV2: 256 5x5 filters at stride 1, pad 2

[13x13x256] MAX POOL2: 3x3 filters at stride 2

[13x13x256] NORM2: Normalization layer

[13x13x384] CONV3: 384 3x3 filters at stride 1, pad 1

[13x13x384] CONV4: 384 3x3 filters at stride 1, pad 1

[13x13x256] CONV5: 256 3x3 filters at stride 1, pad 1

[6x6x256] MAX POOL3: 3x3 filters at stride 2

[4096] FC6: 4096 neurons

[4096] FC7: 4096 neurons

[1000] FC8: 1000 neurons (class scores)

VGG

[Simonyan and Zisserman 2014]

Only 3x3 CONV stride 1, pad 1
and 2x2 MAX POOL stride 2

In one paper VGG-16 was best.
In others VGG-19 is best.

11.2% top 5 error in ILSVRC 2013

->

7.3% top 5 error

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224×224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

Hyper-parameter Search

Many hyper-parameters:

Network Architecture

depth

breadth

layer types

window sizes

strides

...

Training

optimizer type

learning rate

weight decay

initialization

loss function

...

Weight Initialization

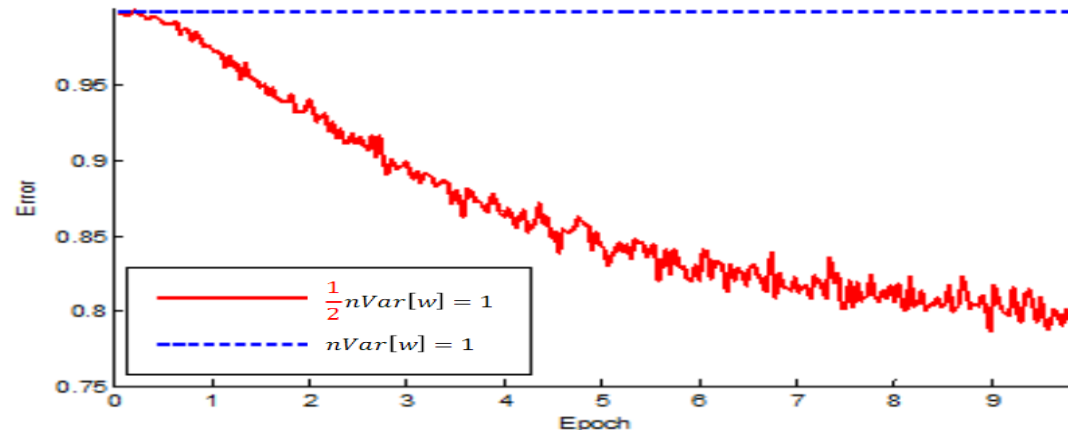
Option 1: Random

Weights selected randomly from a distribution that depends on number of connections (“Xavier”, “MSRA”)

Option 2: Transfer Learning

Use weights that were trained on the same network on another task.

30-layer ReLU net:
good init is able to converge



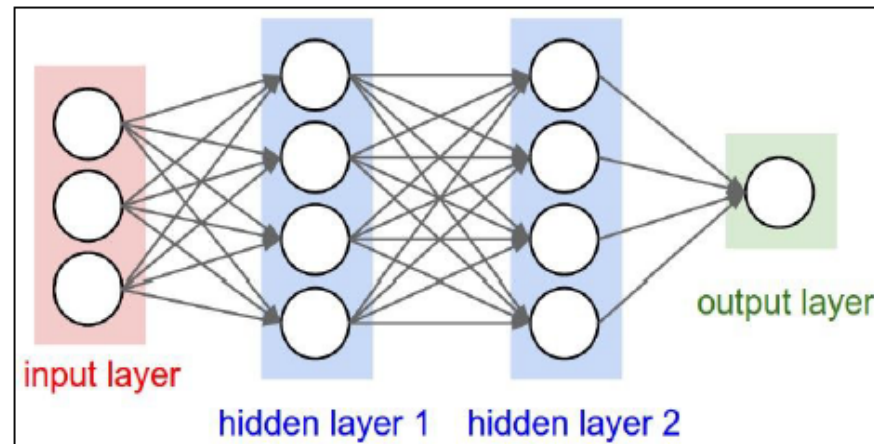
Epoch – One pass over all training set

Mini-batch SGD

Mini-batch **Stochastic Gradient Descent (SGD)**:

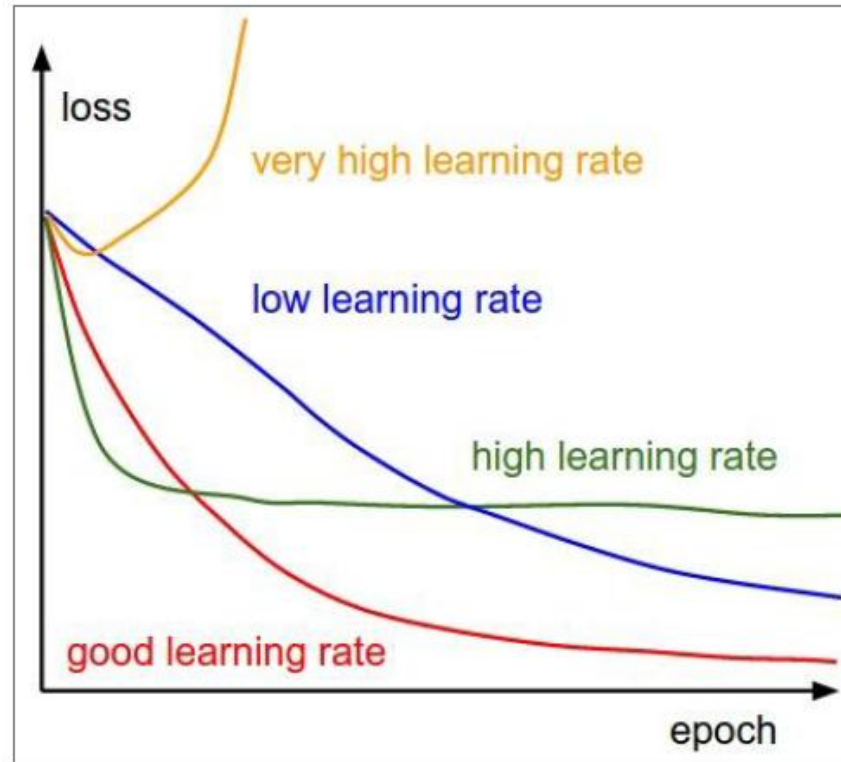
Loop:

1. **Sample** a batch of data
2. **Forward** prop it through the graph, get loss
3. **Backprop** to calculate the gradients
4. **Update** the parameters using the gradient



Learning Rate

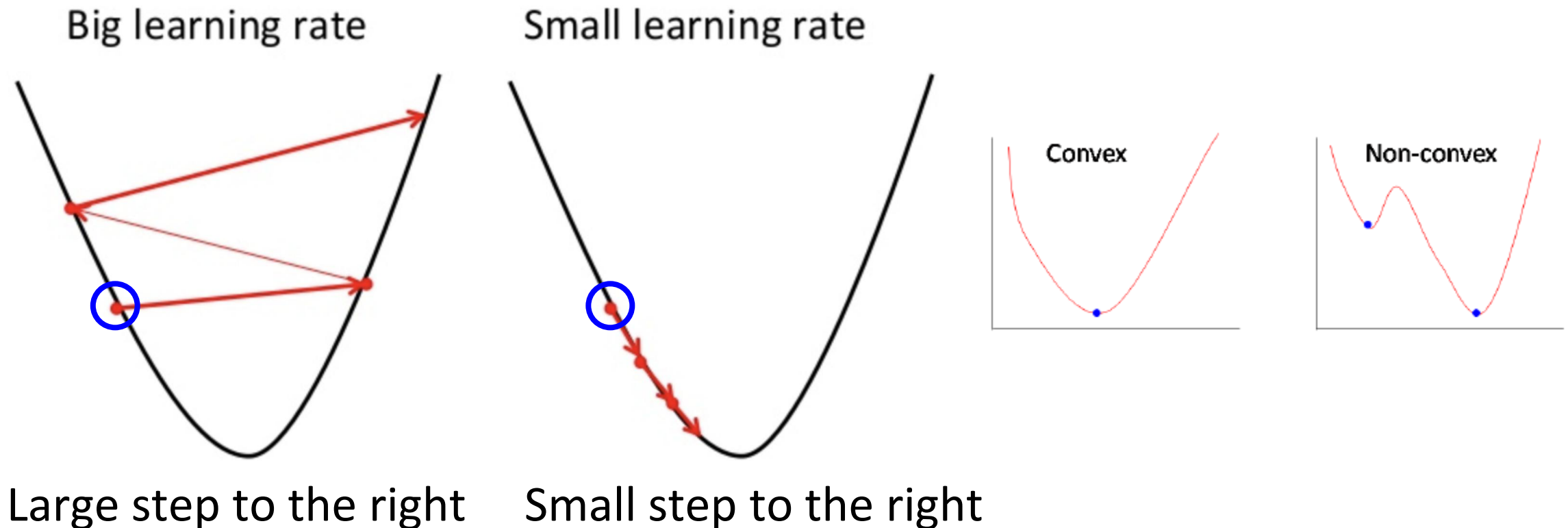
All variants of SGD have learning rate as a hyper-parameter, and properly setting it is **crucial**



Epoch – One pass over all training set

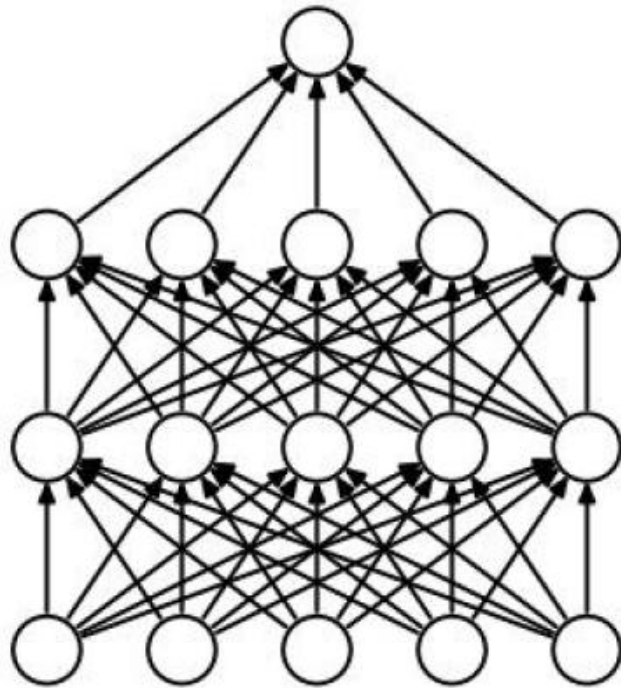
Learning Rate – Simple Example

- Minimization of a Mean Squared Error (MSE)
- Take a step in the gradient direction
- Learning rate – step size

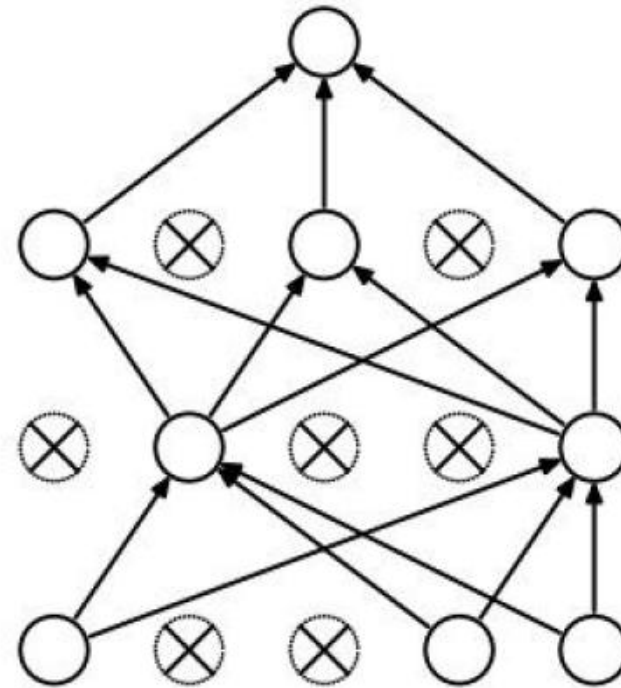


Dropout Regularization (Avoid Overfitting)

Randomly set some neurons to 0 in forward pass:



(a) Standard Neural Net



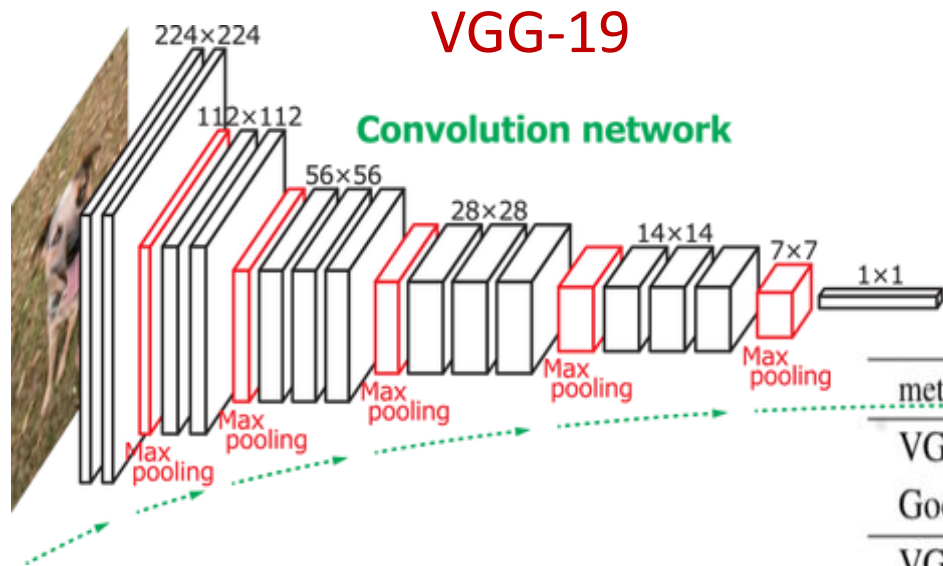
(b) After applying dropout.

[Srivastava et al. 2014]

Preventing Overfitting

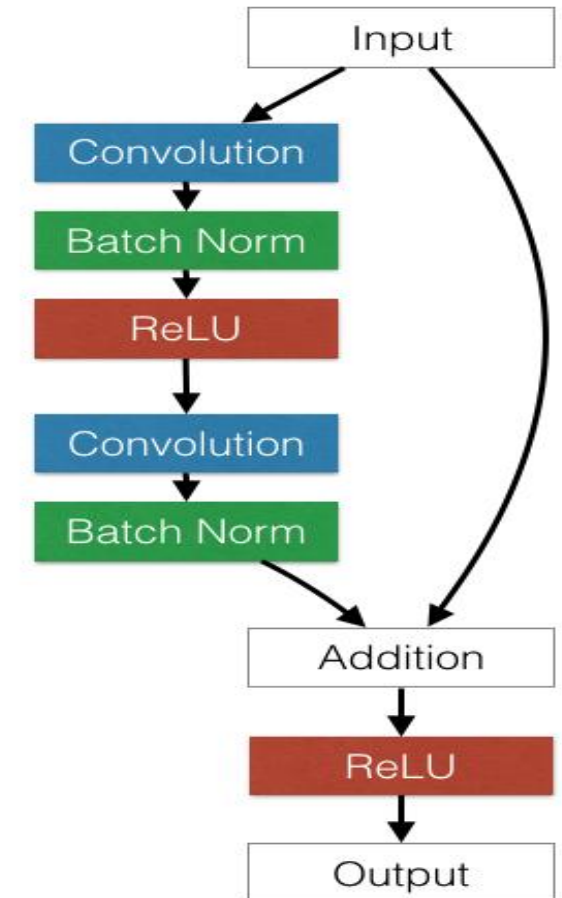
- Regularization term in Loss function
- Dropout Regularization
- Adding Batch Normalization layers
 - Run all examples in batch, and collect all activations of a neuron for this batch.
 - **Normalize the activations of that neuron by subtracting the mean and dividing by standard deviation**
 - Do this for all neurons in the layer.

Computer Vision - Object Category Recognition



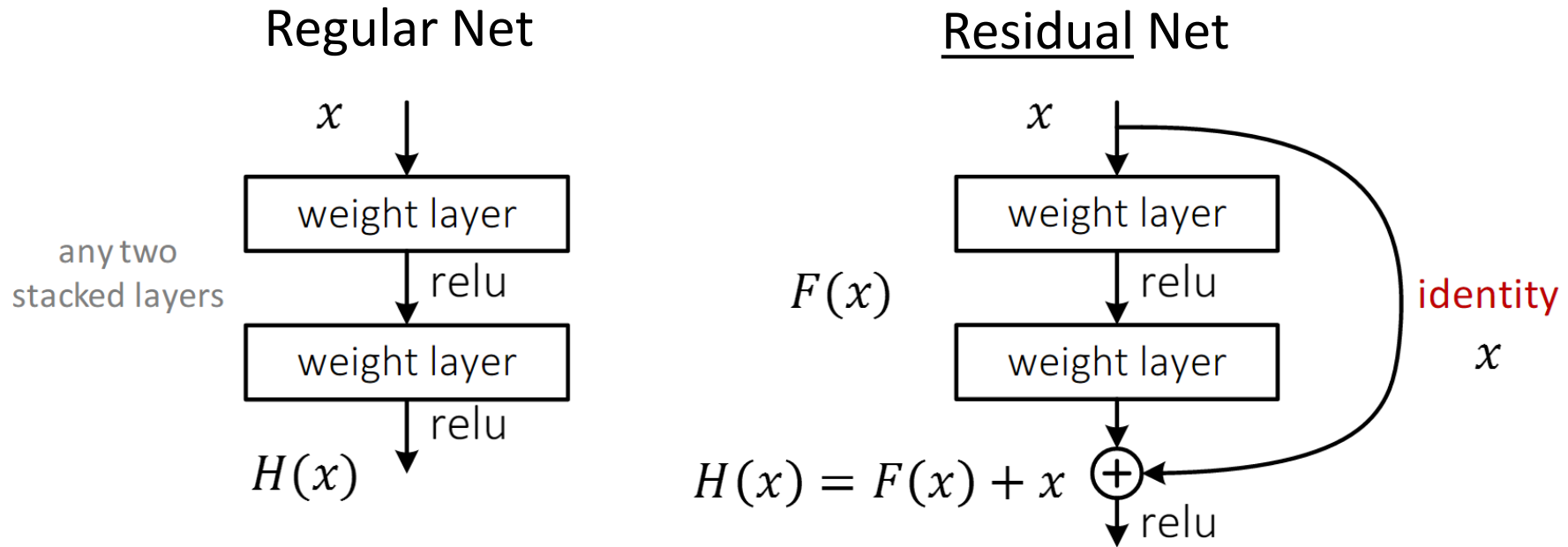
method	top-1 err.	top-5 err.
VGG [41] (ILSVRC'14)	-	8.43 [†]
GoogLeNet [44] (ILSVRC'14)	-	7.89
VGG [41] (v5)	24.4	7.1
PReLU-net [13]	21.59	5.71
BN-inception [16]	21.99	5.81
ResNet-34 B	21.84	5.71
ResNet-34 C	21.53	5.60
ResNet-50	20.74	5.25
ResNet-101	19.87	4.60
ResNet-152	19.38	4.49

Table 4. Error rates (%) of **single-model** results on the ImageNet validation set (except [†] reported on the test set).



ResNet - 1000 layers!

ResNet

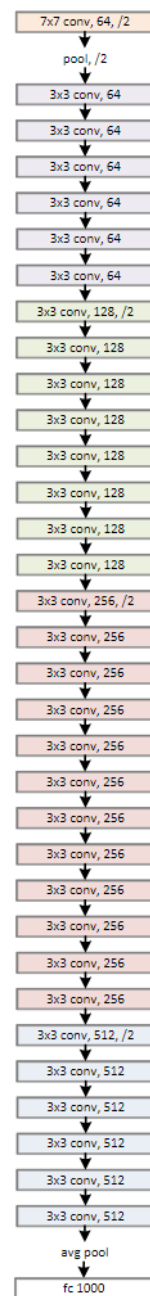


- Instead of learning $H(x)$ from x , network learns the difference $H(x) - x$

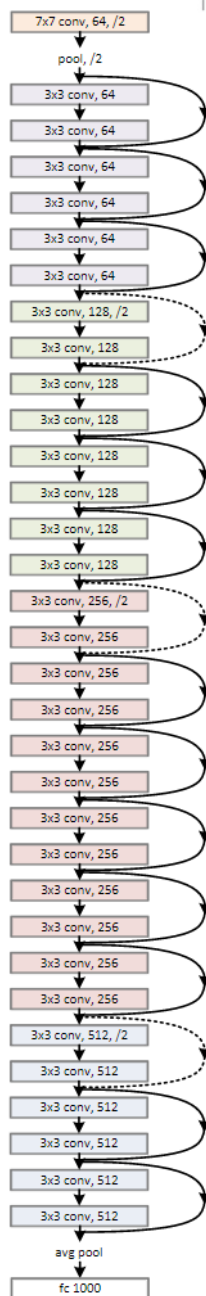
Network “Design”

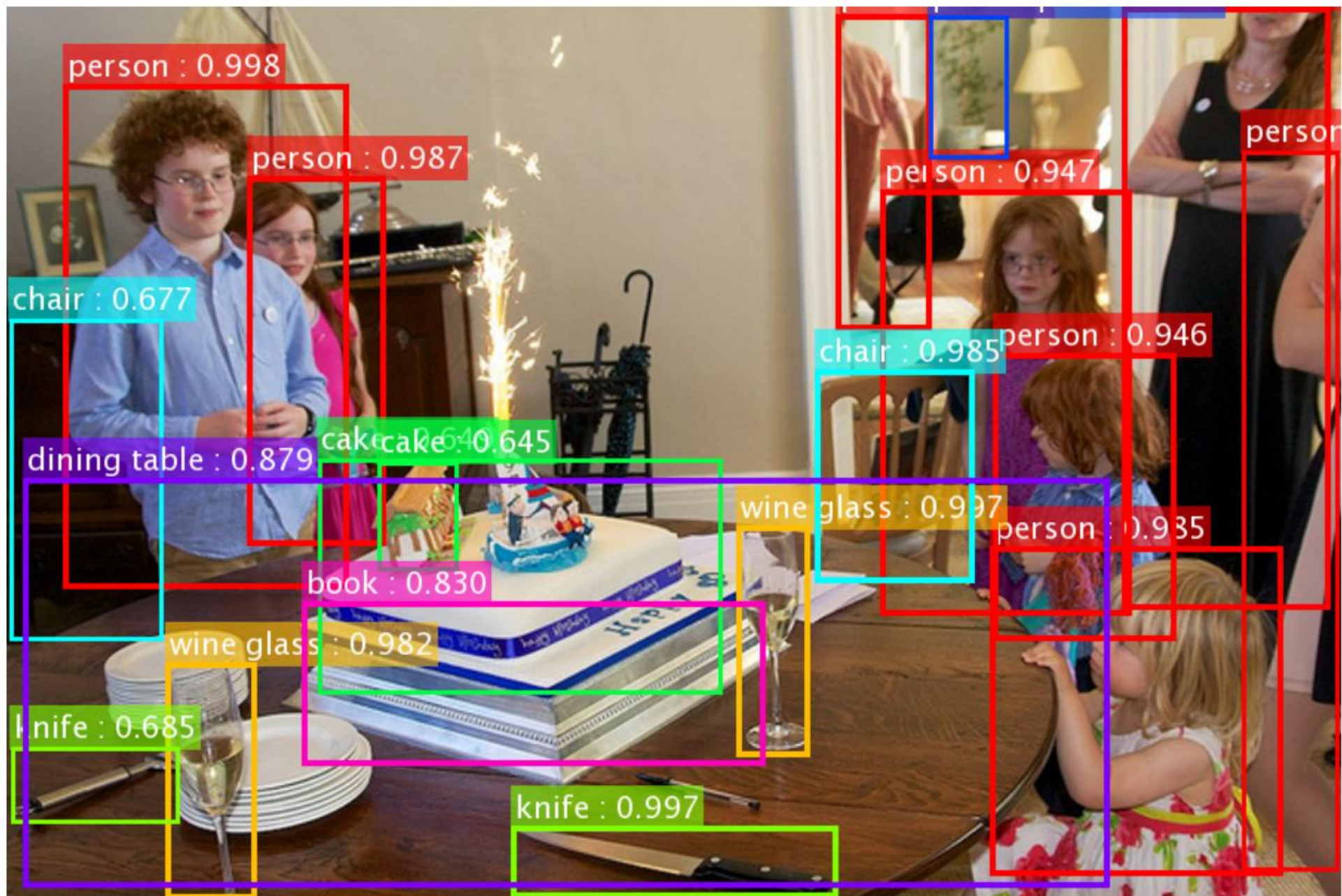
- Keep it simple
- Our basic design (VGG-style)
 - all 3x3 conv (almost)
 - spatial size /2 => # filters x2
 - Simple design; just deep!
- Other remarks:
 - no max pooling (almost)
 - no hidden fc
 - no dropout

plain net



ResNet





ResNet's object detection result on COCO

NN Applications

LLM: Conversations about anything
Automatic Speech & Text Recognition
Image and Video Generation

Computer Vision: Cool Applications



Style Transfer (App: Prisma)

A Neural Algorithm of Artistic Style, L. Gatys , A. Ecker, M. Bethge (Tubingen), <http://arxiv.org/abs/1508.06576>



Example – Student's Phone App

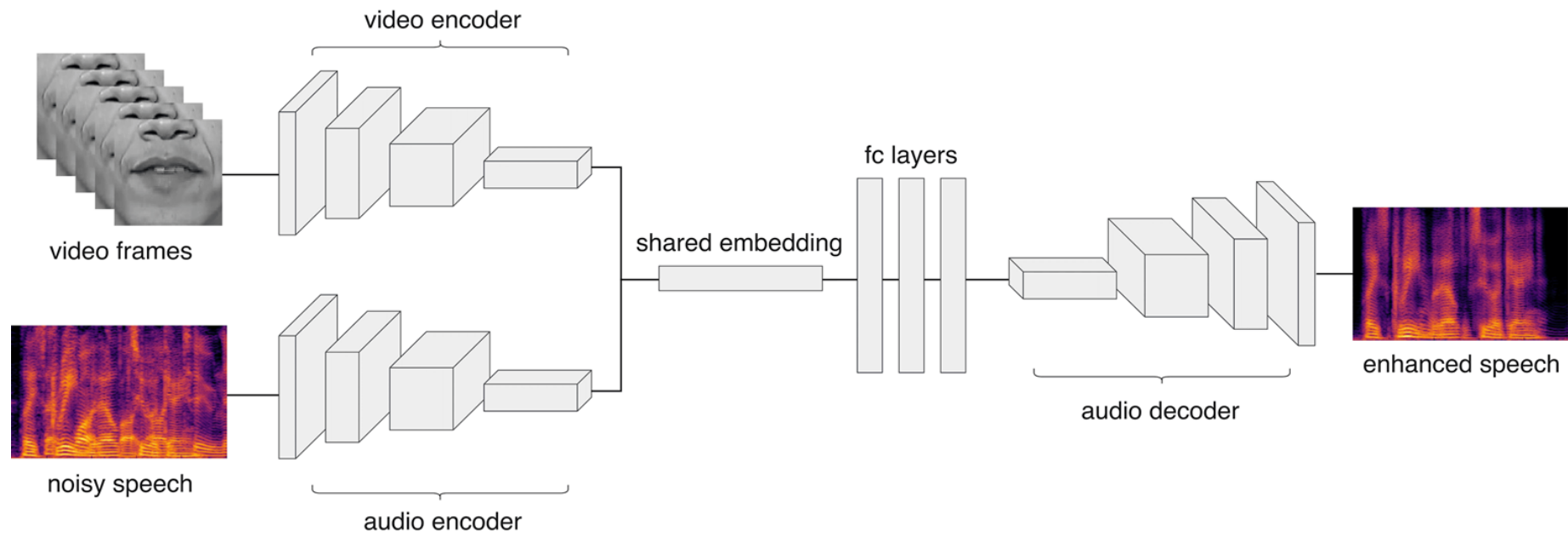


Example – Student's Phone App



End-To-End Speech Enhancement Network

- Input 1: 5 consecutive video frames showing mouth
- Input 2: Spectrogram of noisy speech
- Output: Spectrogram of enhanced speech



Google Example (Same student...): Input video



Only the speaker on the left

